《NetApp 服务引擎 API 参考》

Keystone

NetApp
May 04, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/keystone/seapiref_authorization_and_authentication.html on May 04, 2022. Always check docs.netapp.com for the latest.

目录

NetApp 服务引擎 API 概述 1
授权和身份验证 2
NetApp 服务引擎 REST API 3
使用者 API 4
管理员 API 59

NetApp 服务引擎 API 概述

NetApp 服务引擎提供了一组 API ，用于通过 RESTful Web 服务界面管理受支持存储系统上的存储资源，以实现任何第三方集成。这些 API 涵盖了 GUI 支持的所有功能。NetApp 提供了 API 文档，但客户有责任将其集成到其应用程序中。

本《 NetApp 服务引擎 API 参考指南》为您提供了有关 API 和示例代码的信息。通过本指南中提供的信息，您可以为 NetApp 服务引擎创建 RESTful 客户端。API 基于表述性状态传输（ Representational State Transfer ，REST ）架构模式。

目标受众

本指南面向创建通过 REST API 与 NetApp 服务引擎软件交互的应用程序的开发人员。

如果要使用存储提供程序， NetApp ONTAP 集群和管理管理 API 来管理存储，应使用本指南。

NetApp 服务引擎 API 访问和类别

构建 URL 以直接访问 REST API

您可以直接通过编程语言访问 REST API ，例如 Python ， C# ， C++ ， JavaScript ， 等等。要以 `https ：// <hostname>/API` 格式访问 REST API ，请输入主机名或 IP 地址以及 URL 。

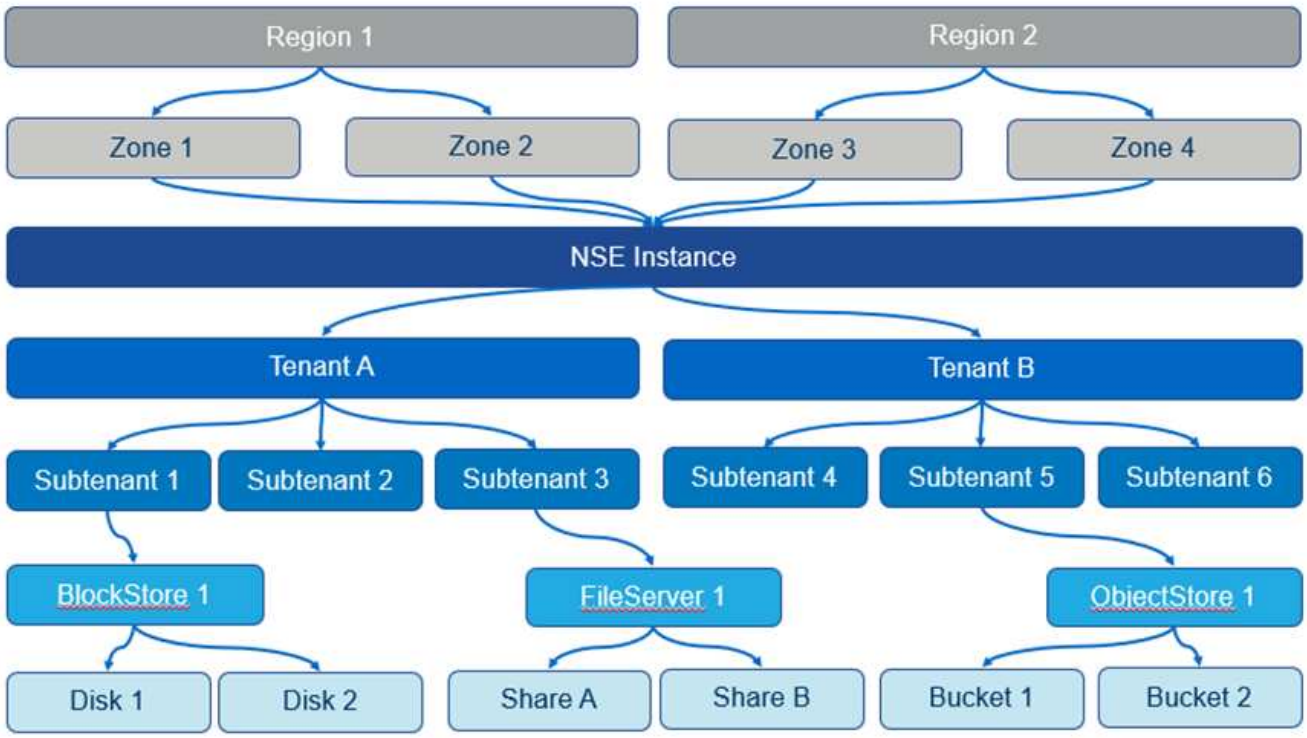
使用者和管理员 API

本文档将 API 说明分为两部分，其中包括：

- 使用者 API
- 管理员 API

NetApp 服务引擎的主要概念

NetApp 服务引擎支持区域和区域的概念，如下图所示。区域表示数据中心或站点，而区域表示区域内的存储子单元（从技术上讲，是指数据中心 / 站点内的集群）。多个分区支持数据可用性和数据保护（ DP ）功能。



一个 NetApp 服务引擎实例可以支持一个或多个租户。NetApp 服务引擎将租户和子租户概念用作拥有或管理存储服务的分层实体。

租户可以是客户，合作伙伴或计费实体。一个租户持有 Flex 订阅服务的订阅（或多个订阅）。

子租户是完全位于租户内的实体。它可用于回放，安全隔离等。

存储项按子租户创建。每个子租户都可以拥有适合该子租户的类型和编号的多个存储项。

NetApp 服务引擎支持以下类型的存储：

- 块存储和磁盘中的块存储
- 文件服务器和文件共享中的文件存储
- 存储在存储分段中

授权和身份验证

NetApp 服务引擎 REST API 可通过任何可对 HTTP 请求进行问题描述处理的 Web 浏览器或编程平台进行访问。NetApp 服务引擎支持基本的 HTTP 身份验证机制以及 Java Web 令牌（ JWT ）承载身份验证。在调用 API 之前，您必须对用户进行身份验证。

HTTP status codes

在运行 API 或解决问题时，您应了解 NetApp 服务引擎 API 使用的各种 HTTP 状态代码和错误代码。

下表列出了与身份验证相关的错误代码。

HTTP 状态代码	状态代码标题	Description
200	确定	成功执行同步 API 调用时返回。
201	已创建	通过同步调用创建新资源，例如配置 Active Directory 。
202	已接受	成功执行异步调用以执行配置功能（例如创建 LUN 和文件共享）时返回。
400	请求无效	指示输入验证失败。用户必须更正输入，例如，请求正文中的有效密钥。
401	未授权请求	您无权查看此资源 / 未授权。
403	已禁止请求	禁止访问您尝试访问的资源。
404	未找到资源	未找到您尝试访问的资源。
405.	不允许使用此方法	不允许使用此方法
412	前提条件失败	未满足一个或多个前提条件。
500	内部服务器错误	内部服务器错误。无法从服务器获取响应。此内部服务器错误可能是永久性的，也可能不是永久性的。例如，如果您运行 GET 或 GET ALL 操作并收到此错误，建议您重复此操作至少五次重试。如果是永久性错误，则返回的状态代码仍为 500 。如果操作成功，则返回的状态代码为 200 。

使用 **API** 的建议

使用 API 时，应遵循某些建议的做法：

- 要有效执行，请确保所有响应内容类型采用以下格式：

```
application/json
```

- 更新阵列值时，必须更新整个值字符串。您不能将值附加到阵列—您只能替换现有阵列。
- 通过组合使用筛选器运算符通配符（` *` ）和管道（`` ）来避免查询对象。它可能检索的对象数不正确。

NetApp 服务引擎 REST API

分页

许多 API 调用返回大量记录。可以通过指定 `offset` 和 `limit` 值来限制记录数：

- * 限制。 * 要返回的记录数。

- * 偏移。 * 返回第一个记录之前要跳过的记录数。例如，如果偏移 1 ，则会跳过第一个记录，并返回从 2 开始的记录。

您还可以对响应进行排序：

- * 排序方式 * 使用指定属性对记录进行排序。例如，要按创建日期进行排序，请指定已创建。
- * order_by-* 按升序（ asc ）或降序（ desc ）顺序排列记录。

使用者 API

概述

本节介绍以下使用者 API：

- 云业务流程 API
- 块存储 API
- 文件服务 API
- 对象存储 API
- 报告 API
- （使用者）管理 API

云编排 API

Cloud Orchestration API NetApp 服务引擎支持将 Cloud Volumes 服务连接到 NetApp 服务引擎，并可在 NetApp 服务引擎中使用这些服务。Cloud Volumes 服务支持 Azure NetApp Files （ANF ）和 Google Cloud Platform （ GCP ）。

Cloud Volumes Services 的订阅在 NetApp 服务引擎之外进行管理。NetApp 服务引擎可通过相关凭据连接到云服务。

本节介绍以下 API：

- 管理 Cloud Volumes Service 实例
- 管理 Azure NetApp Files 帐户
- 管理 Azure NetApp Files 容量池
- 管理 Azure NetApp Files 卷
- 管理 Google Cloud Volumes

Cloud Volumes Service 实例

使用下表中列出的方法检索和创建 Cloud Volumes Service 实例。

HTTP 方法	路径	Description
获取	` v2.1/vcs/instances`	检索所有 Cloud Volumes Service 实例。
发布	` v2.1/vcs/instances`	创建新的 Cloud Volumes Service 实例。
获取	` v2.1/vcs/instances/ ｛ id ｝ `	按 ID 检索 Cloud Volumes Service 实例。
获取	` /v2.1/anf/ 帐户`	检索 Azure NetApp Files 帐户。
发布	` /v2.1/anf/ 帐户`	创建 Azure NetApp Files 帐户
获取	` /v2.1/anf/accouns/ ｛ id ｝ `	按 ID 检索 Azure NetApp Files 帐户。
获取	` v2.1/anf/Pools`	检索所有容量池。
发布	` v2.1/anf/Pools``	创建容量池。
获取	` /v2.1/anf/pools/ ｛ id ｝ `	按 ID 检索 Azure NetApp Files 容量池。

Cloud Volumes Service 实例属性

下表列出了 Cloud Volumes Service 实例属性。

属性	Type	Description
`id`	string	Cloud Volumes Service 实例的唯一标识符。
名称	string	Cloud Volumes Service 实例的名称。
`cc_working _env`	string	Cloud Manager 中的工作环境名称。
`cc_client_id`	string	Auth0 服务 ID 。
`cc_refresh_token`	string	Auth0 刷新令牌。
`cc_account_id`	string	Cloud Central 帐户 ID 。
`cc_account_name`	string	Cloud Central 帐户名称。
`cc_credentials_id`	string	以云为中心的工作环境的 ID 。
`azure_client_id`	string	应用程序（客户端） ID 。Cloud Manager 可用于向 Azure Active Directory 进行身份验证的 Active Directory 服务主体的 ID 。
`azure_client_secret`	string	服务主体应用程序的客户端密钥值。
`azure_tenant_id`	string	Azure Active Directory ID 。这也称为租户 ID 。
`azure_subscription_id`	string	Azure Active Directory 订阅 ID 。

属性	Type	Description
`ssubtenant_id`	string	子租户 ID 。
标记		键值对。

检索 Cloud Volumes Service 实例

HTTP 方法	路径	Description	路径参数
获取	` v2.1/vcs/instances`	检索 Cloud Volumes Service 实例。	`tenant_id` ： （可选）返回属于指定租户的 Cloud Volumes Service 实例。另请参见 "通用分页" parameters

所需的请求正文参数： `none`

按 ID 检索 Cloud Volumes Service 实例

使用下表中列出的方法按标识符检索 Cloud Volumes Service 实例。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/vcs/instances/ { id } `	按 ID 检索 Cloud Volumes Service 实例。	`id` （字符串）： Cloud Volumes Service 实例的唯一标识符。

所需的请求正文属性： `none`

创建 Cloud Volumes Service 实例

使用下表中列出的方法创建新的 Cloud Volumes Service 实例。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/vcs/instances`	创建 Cloud Volumes Service 实例。	无

所需的请求正文属性： `name` ， `cc_working _env` ， `cc_client_id` ， `cc_refresh_token` ， `cc_account_id` ， `cc_account_name` ， `azure_client_id` ， `azure_client_secret` ， `azure_tenant_id` ， `azure_subscription_id` ， `subtenant_id`

- 请求正文示例： *

```
{
  "name": "instance1",
  "cc_working_env": "my-working-env",
  "cc_client_id": "Mu0V1ywgYteI6w1MbD15fKfVIUrNXGWC",
  "cc_refresh_token": "y1tMw3lNzE8JL9jtiE29oSRxOAzYu0cdnwS_2XhjQBr9G",
  "cc_account_id": "account-335jdf32",
  "cc_account_name": "my-account-name",
  "cc_credentials_id": "d336c449-aeb8-4bb3-af28-5b886c40dd00",
  "azure_client_id": "53ba6f2b-6d52-4f5c-8ae0-7adc20808854",
  "azure_client_secret": "NMubGVcDqkwwGnCs6fa01tqlkTisfUd4pBBYgcxxx=",
  "azure_tenant_id": "53ba6f2b-6d52-4f5c-8ae0-7adc20808854",
  "azure_subscription_id": "1933a261-d141-4c68-9d6c-13b607790910",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "tags": {
    "key1": "Value 1",
    "key2": "Value 2",
    "key3": "Value 3",
    "keyN": "Value N"
  }
}
```

管理 Cloud Volumes Service 实例的标记

使用下表中列出的方法为命名的 Cloud Volumes Service 实例指定标记。

HTTP 方法	路径	Description	Parameters
发布	`v2.1/vcs/instances/｛id｝/tags`	管理 Cloud Volumes Service 实例的标记。	`id（string）`：Cloud Volumes Service 实例的唯一标识符。

所需的请求正文属性： `key-value pairs`

- 请求正文示例： *

```
{
  "env": "test"
}
```

Azure NetApp Files 帐户

Azure NetApp Files 帐户属性

下表列出了 Azure NetApp Files 帐户属性。

属性	Type	Description
`id`	string	Azure NetApp Files 帐户的唯一标识符。
名称	string	Azure NetApp Files 帐户的名称。
`resource_group`	string	Azure 资源组。
位置	string	Azure 位置（区域 / 区域）。
`cvs_instance_id`	string	Cloud Volumes Service 实例标识符。
标记	–	键值对。

检索 **Azure NetApp Files** 帐户

HTTP 方法	路径	**Description**	路径参数
获取	` /v2.1/anf/ 帐户`	检索 Azure NetApp Files 帐户。	`ssubtenant_id` ：（强制） Azure NetApp Files 帐户所属的子租户 ID 。`tenant_id` ：（可选）返回属于指定租户的 Azure NetApp Files 帐户。另请参见 "通用分页" parameters

所需的请求正文参数： `none`

按名称检索 **Azure NetApp Files** 帐户

使用下表中列出的方法按名称检索 Azure NetApp Files 帐户。

HTTP 方法	路径	**Description**	**Parameters**
获取	` /v2.1/anf/accouns/ ｛ name ｝ `	按名称检索 Azure NetApp Files 帐户。	`name` （ `string` ）：（强制） Azure NetApp Files 帐户的名称。`ssubtenant_id` （字符串）：（强制） Azure NetApp Files 帐户所属的子租户 ID 。

所需的请求正文属性： `none`

创建 **Azure NetApp Files** 帐户

使用下表中列出的方法创建新的 Azure NetApp Files 帐户。

HTTP 方法	路径	**Description**	**Parameters**
发布	` /v2.1/anf/ 帐户`	创建新的 Azure NetApp Files 帐户。	无

所需的请求正文属性： `name` ， `resource_group` ， `location` ， `cvs_instation_id`

- 请求正文示例： *

```
{
  "name": "string",
  "resource_group": "string",
  "location": "string",
  "cvs_instance_id": "5d2fb0fb4f47df00015274e3",
  "tags": {
    "key1": "Value 1",
    "key2": "Value 2",
    "key3": "Value 3",
    "keyN": "Value N"
  }
}
```

Azure NetApp Files 容量池

容量池属性

下表列出了容量池属性。

属性	Type	Description
`id`	string	容量池的唯一标识符。
名称	string	容量池的名称。
`resource_group`	string	Azure 资源组。
位置	string	Azure 位置（区域 / 区域）。
s大小	整型	容量池的大小（以 TB 为单位）。
`sservice_level`	string	适用的服务级别名称： " 超 " ， " 高级 " 或 " 标准 " 。
`anf_account_name`	string	Azure NetApp Files 帐户实例标识符。
`ssubtenant_id`	string	子租户 ID 。
标记	–	键值对。

检索容量池

HTTP 方法	路径	Description	路径参数
获取	` v2.1/anf/Pools`	检索容量池。	`ssubtenant_id` ：（强制） ANF 帐户所属的子租户 ID 。`tenant_id` ：（可选）返回属于指定租户的容量池。另请参见 "通用分页" parameters

所需的请求正文参数： `none`

- 请求正文示例： *

```
none
```

按名称检索容量池

使用下表中列出的方法按名称检索容量池。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/anf/pools/ ｛name｝`	按名称检索容量池。	`name` （ `string` ）：（强制）容量池的唯一名称。`ssubtenant_id` （字符串）：（强制）容量池所属的子租户 ID 。

所需的请求正文属性： `none`

创建容量池

使用下表中列出的方法创建新的容量池。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/anf/Pools`	创建容量池。	无

所需的请求正文属性： `name` ， `resource_group` ， `location` ， `size` ， `service_level` ， `anf_account_name` ， `subtenant_id`

- 请求正文示例： *

```
{
  "name": "string",
  "resource_group": "string",
  "location": "string",
  "size": 10,
  "service_level": "Standard",
  "anf_account_name": "myaccount",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "tags": {
    "key1": "Value 1",
    "key2": "Value 2",
    "key3": "Value 3",
    "keyN": "Value N"
  }
}
```

修改容量池的大小

使用下表中列出的方法修改容量池的大小。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/anf/pools/ { name } `	修改容量池的大小。	name （ string ）：Mandatory ： 容量池的唯一名称。

所需的请求正文属性： name ， resource_group ， location ， anf_account_name ， size ， service_level ， subtenant_id

- 请求正文示例： *

```
{
  "name": "myaccount",
  "resource_group": "string",
  "location": "string",
  "anf_account_name": "myaccount",
  "size": 4,
  "service_level": "Standard",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "tags": {
    "key1": "Value 1",
    "key2": "Value 2",
    "key3": "Value 3",
    "keyN": "Value N"
  }
}
```

Azure NetApp Files 卷

Azure NetApp Files 卷属性

下表列出了 Azure NetApp Files 卷的属性。

属性	Type	Description
id	string	Azure NetApp Files 卷的唯一标识符。
名称	string	Azure NetApp Files 卷的名称。
resource_group	string	Azure 资源组。
ssubtenant_id	string	子租户 ID 。
anf_account_name	string	Azure NetApp Files 帐户名称。
anf_pool_name	string	Azure NetApp Files 池名称。
位置	string	Azure 位置（区域 / 区域）。
file_path	string	创建令牌或文件路径。用于访问卷的唯一文件路径。
quota_size	整型	GiB 允许的最大存储配额。
ssubNetID	string	委派子网的 Azure 资源 URL 。必须具有委派 Microsoft NetApp/volumes 。
标记	–	键值对。

检索 Azure NetApp Files 卷

使用下表中列出的方法检索 Azure NetApp Files 卷。指定 `tenant_id` 仅返回属于该租户的帐户。

HTTP 方法	路径	Description	路径参数
获取	` v2.1/anf/volumes`	检索 Azure NetApp Files 卷。	`ssubtenant_id` ：（强制） ANF 卷所属的子租户 ID 。`tenant_id` ：（可选）返回属于指定租户的 ANF 卷。另请参见 "通用分页" parameters

所需的请求正文参数： `none` 。

按名称检索 Azure NetApp Files 卷

使用下表中列出的方法按名称检索 Azure NetApp Files 卷。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/anf/volumes/ ｛ name ｝ `	按名称检索 Azure NetApp Files 卷。	`name` （ `string` ）： Mandatory： Azure NetApp Files 卷的唯一名称。`ssubtenant_id` ：（字符串）必填。Azure NetApp Files 卷所属的子租户 ID 。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

创建 Azure NetApp Files 卷

使用下表中列出的方法创建新的 Azure NetApp Files 卷。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/anf/volumes`	创建 Azure NetApp Files 卷	无

所需的请求正文属性： `name` ， `resource_group` ， `subtenant_id` ， `anf_account_name` ， `anf_pool_name` ， `virtual_network` ， `location` ， `file_path` ， `quota_size` ， `subNetID`

- 请求正文示例： *

```
{
  "name": "myVolume",
  "resource_group": "string",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "anf_account_name": "myaccount",
  "anf_pool_name": "myaccount",
  "virtual_network": "anf-vnet",
  "location": "string",
  "file_path": "myVolume",
  "quota_size": 100,
  "subNetId": "string",
  "protocol_types": [
    "string"
  ],
  "tags": {
    "key1": "Value 1",
    "key2": "Value 2",
    "key3": "Value 3",
    "keyN": "Value N"
  }
}
```

管理适用于 **Google Cloud** 的 **Cloud Volumes Service**

您可以使用云编排类别下的 ` /v2.1/gcp/volumes` API 来管理 Google Cloud 实例的云卷。运行此 API 之前，请确保已为子租户启用 Google 云平台（ GCP ）订阅的 Cloud Volumes Service 帐户。

HTTP 动词	路径	**Description**	必需参数 / 请求正文
获取	` v2.1/gcp/ 卷`	您可以使用 GET 方法检索为子租户的 Cloud Volumes Service 订阅创建的所有 Google 云卷的详细信息。	`offset`： 开始收集结果集之前要跳过的项数。`limit`： 要返回的项数。`ssubtenant_id`： 订阅 Google Cloud 的子租户的 ID 。`reGion`： 订阅服务的区域。
获取	` /v2.1/gcp/volumes/ ｛ id ｝ `	您可以使用此方法检索为子租户的 Cloud Volumes Service 订阅创建的特定 Google 云卷的详细信息。	`id`： GCP 卷的 ID 。`ssubtenant_id`： 订阅 Google Cloud 的子租户的 ID 。`reGion`： 订阅服务的区域。

HTTP 动词	路径	Description	必需参数 / 请求正文
发布	` v2.1/gcp/ 卷`	为子租户创建 GCP 卷。在请求正文中添加值以使用指定参数创建卷。	` { "subtenant_id" : "<ID>" , "name" : "<Volume_name>" , "zone" : "<zone>" , "zone" : "<zone>" , "creation_token" : "<token>" , "allowed_clients" : "<IP address of the clients allowed to access GCP>" , "network" : "< 为 GCP 订阅服务输入的网络详细信息 >" , "protocol_types" : < 用于连接的协议，例如 NFSv3 >" , "quota_gib" : < 卷配额（以字节为单位），"service_level" , "< 标准服务级别 " 标签 `: < 例如: < 服务级别 >
PUT	` /v2.1/gcp/volumes/ { id } `	修改已为子租户创建的 GCP 卷。在请求正文中添加要修改的卷的卷 ID 以及要修改的参数的值。	` { "subtenant_id" : "<ID>" , "name" : "<Volume_name>" , "zone" : "<zone>" , "zone" : "<zone>" , "allowed_clients" : "<IP address of the clients allowed to access GCP>" , "quota_gib" : <volume quota in bytes>" , "nfas_protocol" , "< 服务级别 " `, < 服务类型 ›，<nfap_level>" , < 服务类型 ›，< 服务类型 < 服务级别 ›: < 服务级别 ›，< 服务级别 > , "< 服务级别 >
删除	` /v2.1/gcp/volumes/ { id } `	您可以使用此方法删除为子租户的 Cloud Volumes Service 订阅创建的特定 Google 云卷。	id： GCP 卷的 ID 。ssubtenant_id：订阅 Cloud Volumes Service for Google Cloud 的子租户的 ID 。reGion：订阅服务的区域。

块存储 API

您可以使用块存储 API 查看和管理块存储。

在创建磁盘之前，必须创建块存储。创建磁盘时，您必须创建主机组或选择现有主机组才能访问该磁盘。有关详细信息，请参见 "使用块存储"。

块存储

您可以使用块存储 API 检索和管理块存储。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`v2.1/blockstore`	您可以检索所有块存储的详细信息。检索块存储的详细信息，例如块存储 ID ， IP 地址，区域，分区，子网 ID ， 和标记。	`offset` ： 开始收集结果集之前要跳过的项数。`limit` ： 要返回的项数。
获取	`/v2.1/blockstores/｛id｝`	您可以检索特定块存储的详细信息。根据输入的 ID 检索块存储的详细信息，例如 IP 地址，区域，分区，子网 ID 和标记。	`id` ： 块存储的 ID 。
发布	`v2.1/blockstore`	您可以创建块存储。在请求正文中添加值以使用指定参数创建块存储，例如服务协议，子租户 ID ，区域，子网 ID 和标记。	`ssubtenant_id` ： 子租户的 ID 。`zone` ： 分区的名称。`ssubnet_id` ： 子网的 ID 。
发布	`/v2.1/blockstores/｛id｝ /` 标记	您可以为块存储创建或替换标记。在请求正文中添加块存储的 ID 以及 "key ： value pair" 格式的标记值。	`id` ： 块存储的 ID 。
`PUT`	`/v2.1/blockstores/｛id｝`	您可以根据任何块存储的 ID 来修改其。添加块存储的 ID 以及要在请求正文中修改的值，例如分区，服务协议和标记。	`id` ： 块存储的 ID 。
d删除	`/v2.1/blockstores/｛id｝`	您可以按 ID 删除任何块存储。	`id` ： 块存储的 ID 。

删除块存储之前，应删除映射到该块存储的所有磁盘。

Disks

您可以使用磁盘 API 检索和管理磁盘。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	` v2.1/disks`	您可以检索所有磁盘的详细信息。检索磁盘的详细信息，例如块存储 ID ，名称，磁盘路径，协议，快照策略， 和标记。	`offset` ： 开始收集结果集之前要跳过的项数。`limit` ： 要返回的项数。
获取	` /v2.1/disks/{ id } `	您可以检索特定磁盘的详细信息。检索磁盘的详细信息，例如块存储 ID ，名称，磁盘路径，协议，快照策略， 和标记。	`id` ： 磁盘的 ID 。
发布	` v2.1/disks`	您可以创建磁盘。在请求正文中添加值以使用指定参数创建磁盘，例如子租户 ID ，分区，名称，磁盘路径，快照策略， 备份策略和标记。	`ssubtenant_id` ： 子租户的 ID 。`zone` ： 分区的名称。`name` ： 磁盘的名称。`disk_path` ： 磁盘的路径。`protocol` ： 用于块设备访问的存储协议。`OS_type` ： 主机操作系统的类型。`hostgroup_mappings` ： 主机组映射。`sservice_level` ： 适用的服务级别名称：标准，高级，高级分层，至尊或至尊分层。`size_GB` ： 磁盘大小，以 GB 为单位。
发布	` /v2.1/disks/{ id } /` `snapshot/{ name }`	您可以创建磁盘快照。	`id` ： 磁盘的 ID 。`name` ： 快照的名称。
发布	` /v2.1/disks/{ id } /` 标记	您可以为磁盘创建或替换标记。在请求正文中以 "key ： value pair" 格式添加磁盘 ID 和标记值。	`id` ： 磁盘的 ID 。
`PUT`	` /v2.1/disks/{ id } `	您可以根据磁盘 ID 修改任何磁盘。添加磁盘的 ID 以及要在请求正文中修改的值，例如名称，服务级别，快照策略，备份策略和标记。	`id` ： 磁盘的 ID 。`hostgroup_mappings` ： 主机组映射。
d删除	` /v2.1/disks/{ id } `	您可以按 ID 删除任何磁盘。	`id` ： 磁盘的 ID 。
d删除	` /v2.1/disks/{ id } /` `snapshot/{ name }`	您可以按磁盘 ID 和快照名称删除磁盘的任何快照。	`id` ： 磁盘的 ID 。`name` ： 快照的名称。

主机组

对磁盘的访问控制通过主机组进行管理。您可以使用主机组 API 检索和管理主机组。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`/v2.1/hostgroups`	您可以检索所有主机组的详细信息。检索主机组的详细信息，例如名称，子租户详细信息，租户详细信息，区域， 协议，启动程序，使用主机组的磁盘以及标记。	`offset`： 开始收集结果集之前要跳过的项数。`limit`： 要返回的项数。
获取	`/v2.1/hostgroups/ ｛id｝`	您可以检索特定主机组的详细信息。检索主机组的详细信息，例如名称，子租户详细信息，租户详细信息，区域， 协议，启动程序，使用主机组的磁盘以及基于输入的 ID 的标记。	`id`： 主机组的 ID 。
发布	`/v2.1/hostgroups`	您可以创建主机组。在请求正文中添加值以使用指定参数创建主机组，例如名称，子租户 ID ，区域，协议， 启动程序和标记。	`name`： 主机组的名称。`ssubtenant_id`： 子租户的 ID 。`zone`： 分区的名称。`protocol`： 用于块设备访问的存储协议。`OS_type`： 主机操作系统的类型。
发布	`/v2.1/hostgroups/ ｛id｝ /标记`	您可以为主机组创建或替换标记。在请求正文中以 "key： value pair" 格式添加主机组的 ID 以及标记的值。	`id`： 主机组的 ID 。
d删除	`/v2.1/hostgroups/ ｛id｝`	您可以按 ID 删除任何主机组。	`id`： 主机组的 ID 。

主机组中的启动程序

您可以使用主机组 API 检索和管理映射到主机组的启动程序。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`/v2.1/hostgroups/ ｛id｝ /启动程序`	您可以检索所有启动程序的详细信息。检索启动程序及其别名。	`id`： 主机组的 ID 。
获取	`/v2.1/hostgroups/ ｛id｝ /启动程序 / ｛ alias ｝`	您可以检索特定启动程序的详细信息。根据输入的 ID 和别名检索启动程序。	`id`： 主机组的 ID 。`alias`： 启动程序的别名。
发布	`/v2.1/hostgroups/ ｛id｝ /启动程序`	您可以为主机组创建启动程序。在请求正文中添加启动程序及其别名的值，以便为主机组创建启动程序。	`id`： 主机组的 ID 。`alias`： 启动程序的别名。`initiator`： 启动程序（ iSCSI 限定名称或 FC WWPN ）。

HTTP 动词	路径	Description	必需参数 / 请求正文
修补程序	`/v2.1/hostgroups/ {id} /启动程序 / {alias}	您可以修改启动程序。在请求正文中添加新启动程序。	`id`：主机组的 ID 。`alias`：启动程序的别名。`initiator`：启动程序（iSCSI 限定名称或 FC WWPN）。`
d删除	`/v2.1/hostgroups/ {id} /启动程序 / {alias}	您可以按主机组的 ID 和启动程序的别名删除启动程序。	`id`：主机组的 ID 。`alias`：启动程序的别名。

> 将启动程序添加到主机组时，启动程序应与主机组协议匹配。对于采用 iSCSI 协议的主机组，应使用 IQN；对于采用 FC 协议的主机组，应使用 WWPN 。
>
> 从主机组中删除启动程序会影响该主机组映射到的所有磁盘。

文件服务 API

您可以使用文件服务 API 查看和管理文件服务器和文件共享。

在创建文件共享之前，您必须先创建文件服务器来托管它们。

文件服务器 API

您可以使用文件服务器 API 查看和管理文件服务器。有关文件服务器的详细信息，请参见 "创建文件服务器"。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`v2.1/fileservers`	您可以检索所有文件服务器的详细信息。检索文件服务器的详细信息，例如文件服务器 ID ，名称，区域，区域，租户， 和标记。	`offset`：开始收集结果集之前要跳过的项数。`limit`：要返回的项数。
获取	`/v2.1/fileservers/ {id} `	您可以检索特定文件服务器的详细信息。检索文件服务器的详细信息，例如名称，区域，区域，租户， 和标记。	`id`：文件服务器的 ID 。
发布	`v2.1/fileservers`	您可以创建文件服务器。在请求正文中添加值以使用指定参数创建文件服务器，例如名称，子租户 ID ，区域，子网 ID 。	`name`：文件服务器的名称。`ssubtenant_id`：子租户的 ID 。`zone`：分区的名称。`ssubnet_id`：子网的 ID 。

HTTP 动词	路径	Description	必需参数 / 请求正文
发布	`/v2.1/fileservers/｛id｝/标记`	您可以为文件服务器创建或替换标记。在请求正文中以 "key： value pair" 格式添加文件服务器的 ID 以及标记的值。	`id`： 文件服务器的 ID 。
`PUT`	`/v2.1/fileservers/｛id｝`	您可以根据文件服务器的 ID 修改该服务器。添加文件服务器的 ID 以及要在请求正文中修改的值，例如名称，协议，区域，区域和标记。	`id`： 文件服务器的 ID 。
`d`删除	`/v2.1/fileservers/｛id｝`	您可以按 ID 删除任何文件服务器。	`id`： 文件服务器的 ID 。

> 删除文件服务器之前，应删除映射到该服务器的所有文件共享。
>
> 删除启用了 CIFS 的文件服务器后， Active Directory 计算机对象将保留。请 Active Directory 管理员手动从 Active Directory 中删除已删除文件服务器的计算机对象。

文件共享 **API**

您可以使用文件共享 API 查看和管理文件共享。有关文件共享的详细信息，请参见 "创建文件共享"。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`v2.1/fileshares`	您可以检索所有文件共享的详细信息。检索文件共享的详细信息，例如 ID ，名称，Snapshot 策略，协议， 文件服务器 IP 和标记。	`offset`： 开始收集结果集之前要跳过的项数。`limit`： 要返回的项数。
获取	`/v2.1/fileshares/｛id｝`	您可以检索特定文件共享的详细信息。检索文件共享的详细信息，例如 ID ，名称，快照策略，协议，文件服务器 IP ， 和标记。	`id`： 文件共享的 ID 。

HTTP 动词	路径	Description	必需参数 / 请求正文
发布	`v2.1/fileshares`	您可以创建文件共享。在请求正文中添加值以使用指定参数创建文件共享，例如 ID ，名称，Snapshot 策略，协议， 文件服务器 IP 和标记。	`name` ： 文件共享的名称。`share _path` ： 文件共享的路径。`fileserver_id` ： 文件服务器的 ID 。`size_GB` ： 文件共享的大小，以 GB 为单位。`sservice_level` ： 适用的服务级别名称：标准，高级，高级分层，至尊或至尊分层。`protocol` ： 用于访问文件共享的协议（ NFS ， CIFS 或多协议 ）。`security_style` ： 安全模式（ Unix 或 NTFS ）。`export_policy` ： 文件共享的导出策略。
发布	`/v2.1/fileshares/ ｛ id ｝ /snapshot/ ｛ name ｝`	您可以创建文件共享的快照。	`id` ： 文件共享的 ID 。`name` ： 快照的名称。
发布	`/v2.1/fileshares/ ｛ id ｝ /标记`	您可以为文件共享创建或替换标记。在请求正文中以 "key ： value pair" 格式添加文件共享的 ID 和标记值。	`id` ： 文件共享的 ID 。
`PUT`	`/v2.1/fileshares/ ｛ id ｝`	您可以根据文件共享的 ID 修改该文件共享。在请求正文中添加文件服务器的 ID 以及要修改的值，例如名称，协议，快照策略，备份策略和标记。	`id` ： 文件共享的 ID 。`name` ： 文件共享的名称。`size_GB` ： 文件共享的大小，以 GB 为单位。`sservice_level` ： 适用的服务级别名称：标准，高级，高级分层，至尊或至尊分层。`protocol` ： 用于访问文件共享的协议（ NFS ， CIFS 或多协议 ）。`export_policy` ： 文件共享的导出策略。
`d`删除	`/v2.1/fileshares/ ｛ id ｝`	您可以按 ID 删除任何文件共享。	`id` ： 文件共享的 ID 。

HTTP 动词	路径	Description	必需参数 / 请求正文
d删除	` /v2.1/fileshares/ { id } /` snapshot/ { name }	您可以按文件共享的 ID 和快照的名称删除文件共享的任何快照。	id ： 文件共享的 ID 。name ： 快照的名称。

> 对于 CIFS 共享，在共享路径的末尾添加 ` $` 字符将使其成为隐藏共享，例如，pathtomyhiddenshare$ 。

对象存储 API

本节提供用于管理对象存储，对象存储用户和对象存储组的 API 。

对象存储工作流包括以下任务：

- 创建对象存储帐户。
- 创建对象存储组。
- 创建对象存储用户。
- 为用户创建 S3 密钥。

对象存储组

使用下表中列出的方法检索，创建或修改对象存储组。

HTTP 方法	路径	Description
获取	` /v2.1/objectiam/ 组`	检索对象存储组。
获取	` /v2.1/objectiam/groups/ { id } `	按 ID 检索对象存储组。
发布	` /v2.1/objectiam/ 组`	创建对象存储身份访问管理组。
PUT	` /v2.1/objectiam/groups/ { id } `	修改由 ID 标识的对象存储组。
d删除	` /v2.1/objectiam/groups/ { id } `	删除由 ID 标识的对象存储组。

对象存储组属性

下表列出了对象存储属性。

属性	Type	Description
id	string	对象存储组的唯一标识符。
名称	string	对象存储组名称。
s子租户	string	组所属子租户的名称。
ssubtenant_id	string	组所属子租户的标识符。

属性	Type	Description
租户	string	组所属租户的名称。
tenant_id	string	组所属租户的标识符。
s 3_policy		S3 策略，例如： `"s3_policy": { "Statement": [ { "Effect": "Allow", "Action": "s3:*", "Resource": "arn:aws:s3:::*" } ] }`

检索对象存储组

使用下表中列出的方法检索所有对象存储组或部分对象存储组。指定 ssubtenant_id 将仅返回属于该子租户的对象存储组。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/objectiam/ 组`	检索所有对象存储组。或者，指定子租户 ID 以仅检索与子租户关联的对象存储组。	ssubtenant_id （字符串）：与身份和访问管理（IAM）用户 / 组关联的子租户 ID 。offset 和 limit —请参见 "通用分页"

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 1,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 20,
    "records": [
      {
        "id": "5eb1eff8bc5c0300011c989c",
        "name": "MyGroup",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "subtenant": "MySubtenant",
        "s3_policy": {
          "Statement": [
            {
              "Action": [
                "s3:*"
              ],
              "Effect": "Allow",
              "Resource": "arn:aws:s3:::*"
            }
          ]
        }
      }
    ]
```


按 **ID** 检索对象存储组

使用下表中列出的方法按 ID 检索对象存储组。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/objectiam/groups/{id}`	按 ID 检索对象存储组。	`id` （ `string` ）：对象存储组的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5eb1eff8bc5c0300011c989c",
        "name": "MyGroup",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "subtenant": "MySubtenant",
        "s3_policy": {
          "Statement": [
            {
              "Action": [
                "s3:*"
              ],
              "Effect": "Allow",
              "Resource": "arn:aws:s3:::*"
            }
          ]
        }
      }
    ]
  }
```

创建对象存储组

使用以下列出的方法创建对象存储组。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/objectiam/groups/`	创建新的对象存储组服务以托管对象存储用户。	无

所需请求正文属性： `name` ， `ssubtenant_id` ， `s3Policy`

- 请求正文示例： *

```
{
  "name": "MyNewGroup",
  "subtenant_id": "5e7c3af8aab46c00014ce878",
  "s3_policy": {
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "s3:*",
        "Resource": "arn:aws:s3:::*"
      }
    ]
  }
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5fa312c356a0001a73841",
        "action": "create",
        "job_summary": "Create request is successfully submitted",
        "created": "2020-06-02T07:05:21.130260774Z",
        "updated": "2020-06-02T07:05:21.130260774Z",
        "object_id": "5ed5fa312c356a0001a73840",
        "object_type": "sg_groups",
        "object_name": "MyNewGroup",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```


修改对象存储组

使用下表中列出的方法修改对象存储组。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/objectiam/groups/{id} `	修改对象存储组。	id （ string ）：对象存储组的唯一标识符。

所需请求正文属性： name ， ssubtenant_id ， s3Policy

- 请求正文示例： *

```
{
  "s3_policy": {
    "Statement": [
        {
        "Action": [
            "s3:ListAllMyBuckets",
            "s3:ListBucket",
            "s3:ListBucketVersions",
            "s3:GetObject",
            "s3:GetObjectTagging",
            "s3:GetObjectVersion",
            "s3:GetObjectVersionTagging"
        ],
        "Effect": "Allow",
        "Resource": "arn:aws:s3:::*"
        }
    ]
  }
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5fe822c356a0001a73859",
        "action": "update",
        "job_summary": "Update request is successfully submitted",
        "created": "2020-06-02T07:23:46.43550235Z",
        "updated": "2020-06-02T07:23:46.43550235Z",
        "object_id": "5ed5fa312c356a0001a73840",
        "object_type": "sg_groups",
        "object_name": "MyNewGroup",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```


按 **ID** 删除对象存储组

使用下表中列出的方法按 ID 删除对象存储组。

HTTP 方法	路径	**Description**	**Parameters**
d删除	` /v2.1/objectiam/groups/{id}`	按 ID 删除对象存储组。	`id` （ `string` ）：对象存储组的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5eb1eff8bc5c0300011c989c",
        "name": "MyGroup",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "subtenant": "MySubtenant",
        "s3_policy": {
          "Statement": [
            {
              "Action": [
                "s3:*"
              ],
              "Effect": "Allow",
              "Resource": "arn:aws:s3:::*"
            }
          ]
        }
      }
    ]
  }
```


对象存储用户

使用下表中列出的方法执行以下任务：

- 检索，创建或修改对象存储用户。
- 创建 S3 密钥，检索用户的 S3 密钥或按密钥 ID 检索密钥。

HTTP 方法	路径	**Description**
获取	` /v2.1/objectiam/ 用户`	检索对象存储用户。
获取	` /v2.1/objectiam/users/ ｛id｝ `	按 ID 检索对象存储用户。
发布	` /v2.1/objectiam/ 用户`	创建对象存储用户。
PUT	` /v2.1/objectiam/users/ ｛id｝ `	修改由 ID 标识的对象存储用户。

HTTP 方法	路径	Description
d删除	` /v2.1/objectiam/users/ { id } `	按 ID 删除对象存储用户。
获取	` /v2.1/objectiam/users/ { user_id } /s3keys`	获取映射到用户的所有 S3 密钥。
发布	` /v2.1/objectiam/users/ { user_id } /s3keys`	创建 S3 密钥。
获取	` /v2.1/objectiam/users/ { user_id } /s3keys/ { key_id } `	按密钥 ID 获取 S3 密钥。
d删除	` /v2.1/objectiam/users/ { user_id } /s3keys/ { key_id } `	按密钥 ID 删除 S3 密钥。

对象存储用户属性

下表列出了对象存储用户属性。

属性	Type	Description
`id`	string	对象存储用户的唯一标识符。
`display_name`	string	用户的显示名称。
s子租户	string	用户所属子租户的名称。
`ssubtenant_id`	string	用户所属子租户的标识符。
租户	string	用户所属租户的名称。
`tenant_id`	string	用户所属租户的标识符。
`objectiam_user_urn`	string	URN 。
`sg_group_membership`	string	NetApp StorageGRID 组成员资格。例如： "SG_group_membership ： " 5d2fb0fb4f47df00015274e3"]

检索对象存储用户

使用下表中列出的方法检索所有对象存储用户或部分对象存储用户。指定 `ssubtenant_id` 将仅返回属于该子租户的对象存储组。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/objectiam/ 用户`	检索所有对象存储用户。	`ssubtenant_id` （字符串）：与 IAM 用户 / 组关联的子租户 ID 。`offset` 和 `limit` —请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 1,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 20,
    "records": [
      {
        "id": "5eb2212d1cbe3b000134762e",
        "display_name": "MyUser",
        "subtenant": "MySubtenant",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant",
        "objectiam_user_urn":
"urn:sgws:identity::9646563637959535 1967:user/myuser",
        "sg_group_membership": [
          "5eb1eff8bc5c0300011c989c"
        ]
      }
    ]
  }
}
```

按 **ID** 检索对象存储用户

使用下表中列出的方法按 ID 检索对象存储使用情况。

HTTP 方法	路径	**Description**	**Parameters**
获取	` /v2.1/objectiam/users { id } `	按 ID 检索对象存储用户。	`id`： 对象存储帐户 ID 。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5eb2212d1cbe3b000134762e",
        "display_name": "MyUser",
        "subtenant": "MySubtenant",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant",
        "objectiam_user_urn":
"urn:sgws:identity::9646563637959535 1967:user/myuser",
        "sg_group_membership": [
          "5eb1eff8bc5c0300011c989c"
        ]
      }
    ]
  }
}
```

创建对象存储用户

使用下表中列出的方法创建对象存储用户。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/objectiam/ 用户`	创建新的对象存储用户。	无

所需请求正文属性： `display_name` ， `ssubtenant_id` ， `sg_group_membership`

- 请求正文示例： *

```
{
  "display_name": "MyUserName",
  "subtenant_id": "5e7c3af8aab46c00014ce878",
  "sg_group_membership": [
    "5ed5fa312c356a0001a73840"
  ]
}
```

- 响应正文示例：*

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed603712c356a0001a7386c",
        "action": "create",
        "job_summary": "Activate request is successfully submitted",
        "created": "2020-06-02T07:44:49.647815816Z",
        "updated": "2020-06-02T07:44:49.647815816Z",
        "object_id": "5ed603712c356a0001a7386d",
        "object_type": "sg_users",
        "object_name": "MyUserName",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```

修改对象存储用户

使用下表中列出的方法修改对象存储用户。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/objectiam/users/ { id } `	修改由 ID 标识的对象存储用户。	id ： 对象存储用户 ID 。

所需请求正文属性： display_name ， ssubtenant_id ， sg_group_membership

- 请求正文示例： *

```
{
  "display_name": "MyModifiedObjectStorageUser",
  "subtenant_id": "5e57a465896bd80001dd4961",
  "sg_group_membership": [
    "5e60754f9b64790001fe937b"
  ]
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed604002c356a0001a73880",
        "action": "update",
        "job_summary": "Update request is successfully submitted",
        "created": "2020-06-02T07:47:12.205889873Z",
        "updated": "2020-06-02T07:47:12.205889873Z",
        "object_id": "5ed603712c356a0001a7386d",
        "object_type": "sg_users",
        "object_name": "MyUserName",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```


将所有 **S3** 密钥映射到对象存储用户

使用下表中列出的方法将所有 S3 密钥映射到对象存储用户。

HTTP 方法	路径	**Description**	**Parameters**
获取	` /v2.1/objectiam/users/ { user_id } /s3keys`	为对象存储用户创建 S3 密钥。	`user_id` （字符串）：对象存储用户标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e66de2509a74c0001b895e7",
        "display_name": "****************HNDE",
        "subtenant_id": "5e57a465896bd80001dd4961",
        "subtenant": "BProject",
        "objectiam_user_id": "5e66c77809a74c0001b89598",
        "objectiam_user": "MyNewObjectStorageUser",
        "objectiam_user_urn":
"urn:sgws:identity::09936502886898621050:user/mynewobjectstorageuser",
        "expires": "2020-04-07T10:40:52Z"
      }
    ]
```

为对象存储用户创建 **S3** 密钥

使用以下方法为对象存储用户创建 S3 密钥。

HTTP 方法	路径	**Description**	**Parameters**
发布	` /v2.1/objectiam/users/ { user_id } /s3keys`	为对象存储用户创建 S3 密钥。	`user_id` （字符串）：对象存储用户标识符。

所需的请求正文属性： `expires` （字符串）

> 密钥到期日期 / 时间以 UTC 格式设置，必须在将来设置。

- 请求正文示例： *

```
{
  "expires": "2020-04-07T10:40:52Z"
}
```

- 响应正文示例： *

```
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e66de2509a74c0001b895e7",
        "display_name": "****************HNDE",
        "subtenant_id": "5e57a465896bd80001dd4961",
        "subtenant": "BProject",
        "objectiam_user_id": "5e66c77809a74c0001b89598",
        "objectiam_user": "MyNewObjectStorageUser",
        "objectiam_user_urn":
"urn:sgws:identity::09936502886898621050:user/mynewobjectstorageuser",
        "expires": "2020-04-07T10:40:52Z",
        "access_key": "PL86KPEBN6XT4T7UHNDE",
        "secret_key": "FlD/YWAM7JMr9gG8pumU8dzvcTLMzLYtUe2lNzcA"
      }
    ]
  }
}
```


按密钥 **ID** 为对象存储用户获取 **S3** 密钥

使用下表中列出的方法按密钥 ID 为对象存储用户获取 S3 密钥。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/objectiam/users/｛user_id｝/s3keys/｛key_id｝`	按密钥 ID 获取 S3 密钥。	• `user_id` （字符串）：对象存储用户 ID 。例如：5e66c77809a74c0001b89598 • `key_id` （ string ）： S3 密钥，例如：5e66de2509a74c0001b895e7

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ecc7bb9b5d2730001f798fb",
        "display_name": "****************XCXD",
        "subtenant_id": "5e7c3af8aab46c00014ce878",
        "subtenant": "MySubtenant",
        "objectiam_user_id": "5eb2212d1cbe3b000134762e",
        "objectiam_user": "MyUser",
        "objectiam_user_urn":
"urn:sgws:identity::9646563637959535 1967:user/myuser",
        "expires": "2020-05-27T00:00:00Z"
      }
    ]
  }
}
```

按密钥 **ID** 删除 **S3** 密钥

使用下表中列出的方法按密钥 ID 删除 S3 密钥。

HTTP 方法	路径	**Description**	**Parameters**
d删除	` /v2.1/objectiam/users/ { user_id } /s3keys/ { key_id } `	按密钥 ID 删除 S3 密钥。	• `user_id` （字符串）：对象存储用户 ID 。例如：5e66c77809a74c0001b89598 • `key_id` （ `string` ）： S3 密钥，例如：5e66de2509a74c0001b895e7

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content to return for succesful execution
```

对象存储帐户

使用下表中列出的方法执行以下任务：

- 检索，激活或修改对象存储帐户。
- 创建 S3 存储分段。

HTTP 方法	路径	Description
获取	` /v2.1/objectstorage/accounts`	检索对象存储帐户。
获取	` /v2.1/objectstorage/accounts/ { id } `	按 ID 检索对象存储帐户。
发布	` /v2.1/objectstorage/accounts`	创建对象存储帐户。
`PUT`	` /v2.1/objectstorage/accounts/ { id } `	修改由 ID 标识的对象存储帐户。
`d`删除	` /v2.1/objectstorage/accounts/ { id } `	修改由 ID 标识的对象存储帐户。
获取	` /v2.1/objectstorage/b桶`	获取 S3 存储分段。
发布	` /v2.1/objectstorage/b桶`	创建 S3 存储分段。

对象存储帐户属性

下表列出了对象存储帐户属性。

属性	Type	Description
`id`	string	对象存储用户的唯一标识符。
`ssubtenant_id`	string	子租户对象实例的标识符。
`quota_GB`	整型	共享或磁盘的大小。

检索所有对象存储帐户

使用下表中列出的方法检索所有对象存储帐户或部分对象存储帐户。

HTTP 方法	路径	Description	Parameters
获取	`/v2.1/objectstorage/accounts`	检索所有对象存储用户。	`offset` and `limit` – .请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例 *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 19,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 3,
    "limit": 1,
    "records": [
      {
        "id": "5ec6119e6344d000014cdc41",
        "name": "MyTenant - MySubtenant",
        "subtenant": " MySubtenant",
        "subtenant_id": "5ea8c5e083a9f80001b9d705",
        "tenant": "E- MyTenant",
        "tenant_id": "5d914499869caefed0f39eee",
        "sg_account_id": "29420999312809208626",
        "quota_gb": 100,
        "sg_instance_name": "NSE StorageGRID Dev1",
        "sg_instance_id": "5e3ba2840271823644cb8ab6"
      }
    ]
  }
}
```

按 **ID** 检索对象存储帐户

使用下表中列出的方法按 ID 检索对象存储帐户。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/objectstorage/accounts/｛id｝`	按 ID 检索对象存储帐户。	`id`： 对象存储帐户 ID 。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ec6119e6344d000014cdc41",
        "name": "MyTenant - MySubtennant",
        "subtenant": " MySubtennant",
        "subtenant_id": "5ea8c5e083a9f80001b9d705",
        "tenant": " MyTenant",
        "tenant_id": "5d914499869caefed0f39eee",
        "sg_account_id": "29420999312809208626",
        "quota_gb": 100,
        "sg_instance_name": "NSE StorageGRID Dev1",
        "sg_instance_id": "5e3ba2840271823644cb8ab6"
      }
    ]
  }
```

激活对象存储帐户

使用下表中列出的方法激活对象存储帐户。

HTTP 方法	路径	Description	Parameters
发布	`/v2.1/objectstorage/accounts`	激活对象存储服务。	无

所需的请求正文属性： `ssubtenant_id` ， `quota_GB`

- 请求正文示例： *

```
{
  "subtenant_id": "5ecefbbef418b40001f20bd6",
  "quota_gb": 20
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed608542c356a0001a73893",
        "action": "create",
        "job_summary": "Activate request for Sub Tenant MyNewSubtenant is
successfully submitted",
        "created": "2020-06-02T08:05:40.017362022Z",
        "updated": "2020-06-02T08:05:40.017362022Z",
        "object_id": "5ed608542c356a0001a73894",
        "object_type": "sg_accounts",
        "object_name": "MyTenant - MyNewSubtenant",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```

修改对象存储帐户

使用下表中列出的方法修改对象存储帐户。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/objectstorage/accounts/ ｛id｝ `	修改对象存储服务（例如，更改配额）。	id （字符串）：对象存储帐户 ID 。

所需的请求正文属性： name ， ssubtenant_id ， quota_GB

- 请求正文示例： *

```
{
  "name": "MyTenant - MyNewSubtenant",
  "subtenant_id": "5ecefbbef418b40001f20bd6",
  "quota_gb": 30
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed609162c356a0001a73899",
        "action": "update",
        "job_summary": "Update request is successfully submitted",
        "created": "2020-06-02T08:08:54.841652098Z",
        "updated": "2020-06-02T08:08:54.841652098Z",
        "object_id": "5ed608542c356a0001a73894",
        "object_type": "sg_accounts",
        "object_name": "MyTenant - MyNewSubtenant",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```


删除对象存储帐户

在删除对象存储帐户之前，必须先删除所有关联的组，用户和分段。使用下表中列出的方法删除对象存储帐户。

> 使用与 S3 兼容的实用程序删除存储分段。无法从 NetApp 服务引擎中删除存储分段。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/objectstorage/accounts/ ｛id｝ `	删除对象存储帐户。	`id` （字符串）：对象存储帐户 ID 。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "MyTenant - MyNewSubtenant",
  "subtenant_id": "5ecefbbef418b40001f20bd6",
  "quota_gb": 30
}
```

- 响应正文示例：*

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "action": "string",
        "object_id": "5d2fb0fb4f47df00015274e3",
        "object_type": "string",
        "status": "string",
        "status_detail": "string",
        "last_error": "string",
        "user_id": "5d2fb0fb4f47df00015274e3",
        "link": "string"
      }
    ]
  }
}
```

对象存储分段

使用下表中的 API 创建和检索对象存储分段。

HTTP 方法	路径	Description
获取	` /v2.1/objectstorage/b桶`	检索对象存储分段。
发布	` /v2.1/objectstorage/b桶`	创建对象存储分段。

对象存储分段属性

下表列出了对象存储分段属性。

属性	Type	Description
`id`	string	对象存储用户的唯一标识符。
名称	string	存储分段名称。
`ssubtenant_id`	string	存储分段所属子租户的标识符。

检索 **S3** 存储分段

使用下表中列出的方法检索 S3 存储分段。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/objectstorage/b桶`	检索 S3 存储分段。	`ssubtenant_id`： 拥有存储分段的子租户。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "creationTime": "2020-06-02T08:13:25.695Z",
        "name": "mybucket"
      }
    ]
  }
}
```

创建 **S3** 存储分段

使用下表中列出的方法创建 S3 存储分段。

> 在创建存储分段之前，必须存在子租户的对象存储帐户。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/objectstorage/b桶`	创建 S3 存储区。	无

所需的请求正文属性：

- `name` （字符串）： S3 存储分段名称（仅限小写或数字字符）
- `ssubtenant_id` （字符串）： S3 存储分段所属子租户的 ID
- 请求正文示例： *

```
{
  "name": "mybucket",
  "subtenant_id": "5ecefbbef418b40001f20bd6"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Accepted for processing.",
    "verbose_message": "",
    "code": 202
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed60a232c356a0001a7389e",
        "action": "create",
        "job_summary": "Create request is successfully submitted",
        "created": "2020-06-02T08:13:23.105015108Z",
        "updated": "2020-06-02T08:13:23.105015108Z",
        "object_id": "5ed60a232c356a0001a7389f",
        "object_type": "sg_buckets",
        "object_name": "mybucket",
        "status": "pending",
        "status_detail": "",
        "last_error": "",
        "user_id": "5ec626c0f038943eb46b0af1",
        "job_tasks": null
      }
    ]
  }
}
```

备份 API

您可以使用备份 API 查看和管理环境中卷（文件共享和磁盘）的快照（备份或恢复点）。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	`v2.1/backups`	您可以检索为租户下的子租户创建的所有卷的所有备份对象的详细信息。检索源卷和备份对象的详细信息，例如备份策略，分区和标记。	`offset`：开始收集结果集之前要跳过的项数。`limit`：要返回的项数。
获取	`/v2.1/backups/{id}`	您可以检索为租户下的子租户的卷创建的特定备份对象的详细信息。根据输入的 ID 检索源卷和备份对象的详细信息，例如备份策略，分区和标记。	`id`：备份对象的 ID 。
获取	`/v2.1/backup/{id}/recovery_points`	您可以检索特定备份对象的所有恢复点的详细信息。系统将检索详细信息，例如时间戳和名称。	`id`：备份对象的 ID 。
获取	`/v2.1/backup/{id}/recovery_points/{name}`	您可以检索特定备份对象的任何恢复点的详细信息。系统将检索详细信息，例如时间戳和名称。	`id`：备份对象的 ID 。`name`：恢复点的名称。
发布	`v2.1/backups`	您可以为特定源卷创建备份对象。在请求正文中添加值，以使用指定参数创建备份对象，例如备份策略，要保留的备份副本数以及源资源 ID 。	`{ "primary_resource_type": "<资源类型>", "source_resource_id": "<ID>", "backup_zone": "<Backup Zone>", "backup_policy": { "daily_backups_to_keep": <要保留的每日备份数>, "weekly_backups_to_keep": <key_number>, "<only": <保留的每月备份数>, "<key_backups": <key_task_als": <保留的 "<key_als", "<number>: <保留 "<key_backups": <key_als": <key_als", "keep", "keep" "<Tag N>" } }`

HTTP 动词	路径	Description	必需参数 / 请求正文
发布	`/v2.1/backups/｛id｝/tags`	您可以为子租户的备份对象创建或替换标记。在请求正文中以 "key： value pair" 格式添加备份对象的 ID 和标记值。	id：备份对象的 ID。 {"key1"："<标记 1>"，"key2"："<标记 2>"，"keyN"："<标记 N>"}
修补程序	`/v2.1/backups/｛id｝`	您可以根据卷的 ID 修改卷的任何备份对象。添加备份对象的 ID 以及要在请求正文中修改的值，例如备份策略详细信息，要保留的备份数以及源资源 ID。 您可以通过在 patch 请求中将 ssource_resource_id 参数的值设置为空来使备份对象成为孤立对象。此操作将删除 SnapMirror 关系，但会保留备份卷及其恢复点。	id：备份对象的 ID。 {"source_resource_id"："<Resource ID>"，"backup_policy"： {"daily_backups_to_keep"：<要保留的每日备份数>，"weekly_backups_to_keep"：<要保留的每月备份数>，"<key_backups_to_keep"：<要保留的每周备份数>，**<key_task_task_tags：<要保留的每月备份数›**，<"<key_task_task_task_task>，"<"<"<key_task_task_keep"：<月备份数>，"<"<key_task_task_task>：<
d删除	`/v2.1/backups/｛id｝`	您可以删除卷的任何备份对象以及所有恢复点。	id：备份对象的 ID。
d删除	`/v2.1/backup/｛id｝/recovery_points/｛name｝`	您可以删除卷的特定备份对象中的任何恢复点。	id：备份对象的 ID。name：恢复点的名称。

报告 API

使用下表中的方法检索和生成报告。

HTTP 方法	路径	Description
获取	`/v2.1/reports`	检索所有报告列表。
发布	`/v2.1/reports/｛filename｝`	检索报告。

检索报告列表

使用下表中列出的方法检索报告列表。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/reports`	检索租户的所有可用报告。可以按日期范围筛选报告。	• `tenant_id` （字符串） • `sstart` （ `string` ）：检索起始日期指定的报告。 • `end` （ `string` ）：检索在结尾指定日期之前的报告

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "filename": "string"
      }
    ]
  }
}
```

检索通过文件名标识的报告

使用下表中的方法检索文件名标识的报告。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/reports/ { filename } `	检索文件名标识的报告。	`filename` （ `string` ）：报告文件名。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "filename": "string"
      }
    ]
  }
}
```

使用子网 API 定义网络配置

您可以使用子网 API 查看和创建子租户和分区的子网。您可以使用它们定义网络配置。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	` /v2.1/tenant/ { tenant_id } /` 分区 `/ { zone_name } / ` 子网	您可以检索租户和区域的所有子网对象。检索子网的详细信息，例如 VLAN ，子租户，分区，路由， 和标记。	`tenant_id`：租户的 ID 。`zone_name`：分区的名称。`offset`：开始收集结果集之前要跳过的项数。`limit`：要返回的项数。

HTTP 动词	路径	Description	必需参数 / 请求正文
获取	` /v2.1/tenant/ { tenant_id } /` 分区 / { zone_name } /subnets/` { id } `	您可以检索为租户和区域创建的特定子网对象的详细信息。检索子网的详细信息，例如 VLAN ，子租户，分区，路由， 和标记。	tenant_id： 租户的 ID 。zone_name： 分区的名称。id： 子网的 ID 。
发布	` /v2.1/tenant/ { tenant_id } /` 分区 / { zone_name } / 子网	您可以为特定租户和区域创建子网对象。在请求正文中添加值以使用指定参数创建子租户对象，例如名称， VLAN ， CIDR ，子租户 ID ， 路由和标记。	tenant_id： 租户的 ID 。zone_name： 分区的名称。 { "name" ： "string" ， "VLAN" ： "1000" ， "CIDR" ： "10.0.0.0/24 " ， "subtenant_id" ： "5d2fb0fb4f47df00015274e3" ， "routes" ： [{ "destination" ： "10.0.0.0/24 ， "gateway" ： "5d2fb0fb4fb4f47df00015274e3" ， "b" ， "key2" ： " ， "key2" ： " ， "key2" ： "key2" ： " ， "key2" ： "key2" ： " ， "key1" ， "key2" ： "key2" ： "key2" ， "key1" ， "key2" ：
发布	` /v2.1/tenant/ { tenant_id } /` 分区 / { zone_name } /subnets/` { id } /tags`	您可以为租户的子网对象创建或替换标记。在请求正文中添加子网对象的 ID 以及 "key ： value pair" 格式的标记值。	tenant_id： 租户的 ID 。zone_name： 分区的名称。id： 备份对象的 ID 。 { "key1" ： "< 标记 1>" ， "key2" ： "< 标记 2>" ， "keyN" ： "< 标记 N>" }

（使用者）管理 API

（使用者）管理 API 由多种方法组成，可用于执行以下任务：

- 登录，设置密码并刷新身份验证令牌。
- 检索作业并查看作业详细信息。

  请参见 "检索作业" 和 "按作业检索作业 ID"。

- 检索区域。

请参见 "检索区域" 和 "按名称检索区域"。

- 检索服务级别。

  请参见 "检索服务级别" 和 "按名称检索服务级别"。

- 使用子租户。

  请参见

  - "检索所有子租户"
  - "按 ID 检索子租户"
  - "修改子租户"
  - "按 ID 删除子租户"

- 使用子租户。

  请参见

  - "检索所有租户"
  - "按 ID 检索租户"
  - "创建租户"
  - "修改租户"
  - "删除租户"

- 检索用户。

  "检索所有用户"， "按 ID 检索用户"，和 "按用户名检索用户"。

- 检索分区。

  请参见 "检索所有分区" 和 "按名称检索分区"

下表列出了本节中介绍的使用者 API 。

HTTP 方法	路径	Description
发布	` /v2.1/auth/ 密码`	设置用户的密码。
发布	` /v2.1/auth/ 密码`	刷新身份验证 JWT 。
发布	` /v2.1/auth/signin`	登录。
获取	` /v2.1/auth/ 地区`	检索区域。
获取	` /v2.1/auth/Regies/ ｛name｝ `	按名称检索区域。
获取	` /v2.1/auth/zones`	检索分区。
获取	` /v2.1/auth/zones/ ｛name｝ `	按名称检索分区。
获取	` v2.1/jobs/`	检索作业。

HTTP 方法	路径	Description
获取	` /v2.1/jobs/ ｛ id ｝ `	按 ID 检索作业。

重置用户密码

使用下表中列出的方法重置用户密码。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/auth/ 密码`	重置密码。	无

所需的请求正文属性： `username` （字符串）， `new_password` （字符串）

- 请求正文示例： *

```
{
  "username": "MyName",
  "old_password": "oldPassword",
  "new_password": "newPassword"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "user": {
          "id": "5e61aa814559c20001df1a5f",
          "username": "MyName",
          "firstName": "MyFirstName",
          "lastName": "MySurname",
          "displayName": "CallMeMYF",
          "email": "user@example.com",
          "tenancies": [
            {
              "id": "5e5f1c4f253c820001877839",
              "name": "MyTenant",
              "code": "testtenantmh",
              "role": "user"
            }
          ]
        },
        "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Ik15TmFtZSIsImV4cCI6
MTU4MzgxNjg3MX0.ZuRXjDPVtc2pH-e9wqgmszVKCBYS2PLqux2YwQ5uoAM"
      }
    ]
  }
}
```


刷新身份验证令牌

使用下表中列出的方法刷新身份验证令牌。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/auth/refresh`	刷新身份验证令牌。	无

所需的请求正文属性： `none`

- 请求正文示例： `*`

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "user": {
          "id": "5d914547869caefed0f3a00c",
          "username": "myusername",
          "firstName": "myfirstname",
          "lastName": "",
          "displayName": "Myfirstname Mysurname",
          "email": "",
          "tenancies": [
            {
              "id": "5d914499869caefed0f39eee",
              "name": "MyOrg",
              "code": "myorg",
              "role": "admin"
            },
            {
              "id": "5d9417aa869caefed0f7b4f9",
              "name": "ABCsafe",
              "code": "abcsafe",
              "role": "admin"
            }
          ]
        },
        "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImVsbGlvdCIsImV4cCI6
MTU4MzgxNzA2N30.FdKD3QhPoNdWdbMfZ0bzCMTHluIt6HNP311F482K9AY"
      }
    ]
  }
}
```

登录

使用下表中列出的方法登录。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/auth/signin`	以用户身份登录。	无

所需的请求正文属性： `username` （字符串）， `new_password` （字符串）

- 请求正文示例： *

```
{
  "username": "MyName",
  "password": "newPassword"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Authentication succeeeded.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "user": {
          "id": "5e61aa814559c20001df1a5f",
          "username": "MyName",
          "firstName": "MyFirstName",
          "lastName": "MySurname",
          "displayName": "CallMeMYF",
          "email": "user@example.com",
          "tenancies": [
            {
              "id": "5e5f1c4f253c820001877839",
              "name": "MyTenant",
              "code": "testtenantmh",
              "role": "user"
            }
          ]
        },
        "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Ik15TmFtZSIsImV4cCI6
MTU4MzgxNzQwMH0._u_UyYrzg_RewF-9ClIGoKQhfZYWrixZYBrsj1kh1hI"
      }
    ]
  }
}
```


管理员 API

概述

本节介绍以下管理员 API：

- Tenants
- 子租户
- 用户

在使用者 API 套件中，可以执行重置用户密码，刷新令牌或以用户身份登录等活动。请参见（使用者）管理 API 。

- 分区
- regions
- ONTAP 集群
- StorageGRID 实例
- 服务级别
- 服务请求
- 作业

Tenants

使用下表中列出的方法检索，创建，修改和删除租户。

HTTP 方法	路径	Description
获取	` v2.1/tenants`	检索所有租户的列表。
获取	` /v2.1/teners/ ｛ id ｝ `	按租户 ID 检索租户。
发布	` v2.1/tenants`	创建新租户。
`PUT`	` /v2.1/teners/ ｛ id ｝ `	修改租户的详细信息。
d删除	` /v2.1/teners/ ｛ id ｝ `	删除租户。

租户属性

下表列出了租户属性。

属性	Type	Description
`id`	string	租户的唯一标识符。
代码	string	客户指定的（或默认的）代码，用于表示租户。此属性可以包含小写字母，数字和下划线。
名称	string	租户名称。
`zuora_account_name`	string	计费帐户名称： Zuora 中订阅的名称。
`zuora_account_number`	string	计费帐户编号： Zuora 中的订阅编号。
问题描述`	string	租户的问题描述。

属性	Type	Description
使用情况	–	适用于租户的服务和服务详细信息。对于每个服务级别，此属性将显示以下内容： * 名称： * 服务级别名称 * 已用 _size_GB ： * 服务级别名称 * 角色名称： * 用户角色（用户，管理员，读取，配对或 root ）

检索所有租户

使用下表中列出的方法检索所有租户或所有租户的一部分。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/tenants`	检索所有租户。	`offset` 和 `limit` —请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 23,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 2,
    "records": [
      {
        "id": "5e7c3af7aab46c00014ce877",
        "name": "MyTenant",
        "zuora_account_name": "MyAccount",
        "zuora_account_number": "A00000415",
        "description": "",
        "code": "mytenantcode",
```

```
        "usage": {
          "A-S00003875": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 10,
              "burst": 0
            },
            {
              "service_level": "standard",
              "consumed": 1.94,
              "committed": 30,
              "burst": 0
            }
          ],
          "A-S00004566": [
            {
              "service_level": "object",
              "consumed": 3.31,
              "committed": 300,
              "burst": 0
            }
          ]
        }
      },
      {
        "id": "5d914499869caefed0f39eee",
        "name": "MyOrg",
        "zuora_account_name": "MyOrg Inc",
        "zuora_account_number": "A00000415",
        "description": "",
        "code": "myorg",
        "usage": {
          "A-S00003875": [
            {
              "service_level": "standard",
              "consumed": 12.33,
              "committed": 30,
              "burst": 0
            },
            {
              "service_level": "object",
              "consumed": 0,
              "committed": 40,
              "burst": 0
            }
```

```
          ],
          "A-S00003969": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 5,
              "burst": 0
            }
          ]
        }
      }
    ]
  }
}
```

按 ID 检索租户

使用下表中列出的方法按 ID 检索租户。

HTTP 方法	路径	Description	Parameters
获取	`/v2.1/teners/｛id｝`	检索 ID 指定的租户。	`id`（`string`）：租户的唯一标识符。

所需的请求正文属性： `none`

请求正文示例：

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e7c3af7aab46c00014ce877",
        "name": "MyTenant",
        "zuora_account_name": "MyAccount",
        "zuora_account_number": "A00000415",
```

```
        "description": "",
        "code": "mytenantcode",
        "usage": {
          "A-S00003875": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 10,
              "burst": 0
            },
            {
              "service_level": "premium",
              "consumed": 2.4,
              "committed": 20,
              "burst": 0
            },
            {
              "service_level": "standard",
              "consumed": 1.94,
              "committed": 30,
              "burst": 0
            },
            {
              "service_level": "object",
              "consumed": 0,
              "committed": 40,
              "burst": 0
            }
          ],
          "A-S00003969": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 5,
              "burst": 0
            },
            {
              "service_level": "standard",
              "consumed": 0,
              "committed": 30,
              "burst": 0
            }
          ],
          "A-S00004566": [
            {
              "service_level": "object",
```

```
                "consumed": 3.31,
                "committed": 300,
                "burst": 0
              }
            ]
          }
        }
      ]
    }
  }
```

创建租户

使用下表中列出的方法创建租户。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/tenants`	创建新租户。	无

所需请求正文属性： `code` ， `name` ， `zuora_account_name` ， `zuora_account_number`

- 请求正文示例： *

```
{
  "name": "MyNewTenant",
  "code": "mytenant",
  "zuora_account_name": "string",
  "zuora_account_number": "A00000415",
  "description": "DescriptionOfMyTenant"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5ac802c356a0001a735af",
        "name": "MyNewTenant",
        "zuora_account_name": "string",
        "zuora_account_number": "A00000415",
        "description": "DescriptionOfMyTenant",
        "code": "mytenant",
        "usage": null
      }
    ]
  }
}
```

修改租户

使用下表中列出的方法修改租户。

HTTP 方法	路径	Description	Parameters
`PUT`	` /v2.1/teners/ { id } `	修改 ID 指定的租户。您可以更改租户的名称，Zuora 订阅详细信息（帐户名称或订阅编号）和问题描述。	`id` （ `string` ）：租户的唯一标识符。

所需的请求正文属性： `code`

- 请求正文示例： *

```
{
  "name": "MyNewTenant",
  "code": "mytenant",
  "zuora_account_name": "string",
  "zuora_account_number": "A00000415",
  "description": "New description of my tenant"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5ac802c356a0001a735af",
        "name": "MyNewTenant",
        "zuora_account_name": "string",
        "zuora_account_number": "A00000415",
        "description": "New description of my tenant",
        "code": "mytenant",
        "usage": null
      }
    ]
  }
}
```

删除租户

使用下表中列出的方法删除租户。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/teners/ ｛id｝`	删除此 ID 指定的租户。	`id` （ `string` ）：租户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for successful delete
```

子租户

使用下表中列出的方法检索，创建，修改和删除子租户。

HTTP 方法	路径	Description
获取	` /v2.1/ 子租户`	检索子租户。
获取	` /v2.1/subteners/ ｛ id ｝ `	按子租户 ID 检索子租户。
发布	` /v2.1/ 子租户``	创建新的子租户。
`PUT`	` /v2.1/subteners/ ｛ id ｝ `	修改子租户的详细信息。您可以修改子租户的名称。
`d`删除	` /v2.1/subteners/ ｛ id ｝ `	删除子租户。

子租户属性

下表列出了子租户属性。

属性	Type	Description
`id`	string	子租户的唯一标识符。
名称	string	子租户名称。
代码	string	客户指定的（或默认的）代码，表示子租户。
`tenant_id`	string	子租户所属租户的标识符。

检索所有子租户

使用下表中列出的方法检索所有子租户或所有子租户的一部分。指定 `tenant_id` 将仅返回属于该租户的子租户。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ 子租户`	检索子租户。	`tenant_id`：（可选）返回属于指定租户的子租户。偏移和限制—请参见

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 202,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 2,
    "records": [      {
        "id": "5e7c3af8aab46c00014ce878",
        "description": "",
        "name": "MySubtenant",
        "code": "mysubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      },
      {
        "id": "5d9144f3869caefed0f39f82",
        "description": "",
        "name": "Mysubtenant2",
        "code": "myothersubtenant",
        "tenant_id": "5d914499869caefed0f39eee",
        "tenant": "MyTenant"
      }
    ]
  }
}
```


按 ID 检索子租户

使用下面列出的方法按 ID 检索子租户。

HTTP 方法	路径	Description	Parameters
获取	`/v2.1/subteners/{id}`	检索 ID 指定的子租户。	`id`（`string`）：子租户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e7c3af8aab46c00014ce878",
        "description": "",
        "name": "MySubtenant",
        "code": "subtenantcode",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

创建子租户

使用下表中列出的方法创建子租户。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/ 子租户`	创建新的子租户。	无

所需请求正文属性： `name` ， `code` ， `tenant_id`

- 请求正文示例： *

```
{
  "name": "MySubtenant",
  "code": "mynewsubtenant",
  "tenant_id": "5ed5ac802c356a0001a735af"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ecefbbef418b40001f20bd6",
        "description": "",
        "name": "MyNewSubtenant",
        "code": "mynewsubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

按 ID 修改子租户

使用下表中列出的方法按 ID 修改子租户。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/subteners/{id}`	修改 ID 指定的子租户。您可以更改子租户名称。	id（string）：子租户的唯一标识符。

所需的请求正文属性：name

- 请求正文示例：*

```
{
  "name": "MyModifiedSubtenant"
}
```

- 响应正文示例：*

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ecefbbef418b40001f20bd6",
        "description": "",
        "name": "MyNewSubtenant",
        "code": "mynewsubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

按 ID 删除子租户

使用下表中列出的方法按 ID 删除子租户。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/subteners/ ｛id｝ `	删除此 ID 指定的子租户。	`id` （ `string` ）：子租户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

用户

使用下表中列出的方法检索，创建，修改和删除子租户。

HTTP 方法	路径	Description
获取	` v2.1/users`	检索所有用户的列表。
获取	` /v2.1/users/ ｛ id ｝ `	按用户 ID 检索用户。
发布	` v2.1/users`	创建新用户：
PUT	` /v2.1/users/ ｛ id ｝ `	修改用户的详细信息。
d删除	` /v2.1/users/ ｛ id ｝ `	删除用户。
获取	` /v2.1/Users/ ｛ username ｝ `	按用户名检索用户。

用户属性

下表列出了用户属性。

属性	Type	Description
id	string	用户的唯一标识符。
用户名	string	用户名。
密码	string	用户的密码。
名字	string	用户的名字。
lastName	string	用户的姓氏。
displayName	string	用户的显示名称。
电子邮件	string	用户的电子邮件地址。
电话	string	用户的电话号码。
profileImageURL	string	用户配置文件映像的 URL 。
tenant_id	string	此用户的主租户标识符。
租约	–	用户所属的租户；一个数组，包含： • tenant_id ，和 • role_name 这是用户角色；用户，管理员，读取，配对， 或 root 。
提供程序	string	身份验证提供程序： local 或 ActiveDirectory
提供程序数据	–	身份验证提供程序详细信息，包括： • EMail_address ，和 • m成员 （组成员资格）

检索所有用户

使用下表中列出的方法检索所有用户。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/users`	检索所有用户。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5dddbe0ef071fe0001b889fd",
        "username": "TestUser3",
        "firstName": "Test",
        "lastName": "User",
        "displayName": "",
        "email": "testuser@netapp.com",
        "tenancies": [
          {
            "id": "5d914499869caefed0f39eee",
            "name": "MyOrg",
            "code": "myorg",
            "role": "admin"
          }
        ]
      }
    ]
  }
}
```

按 **ID** 检索用户

使用下表中列出的方法按 ID 检索用户。

HTTP 方法	路径	**Description**	**Parameters**
获取	`v2.1/users`	按 ID 检索用户。	`id` （ `string` ）：用户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e585df6896bd80001dd4b44",
        "username": "testuser01",
        "firstName": "",
        "lastName": "",
        "displayName": "",
        "email": "",
        "tenancies": [
          {
            "id": "5d914499869caefed0f39eee",
            "name": "MyOrg",
            "code": "myorg",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

按用户名检索用户

使用下表中列出的方法按用户名检索用户。

HTTP 方法	路径	Description	Parameters
获取	`v2.1/users`	按用户名检索用户。	`username` （ `string` ）：用户的用户名。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61aa814559c20001df1a5f",
        "username": "MyName",
        "firstName": "MyFirstName",
        "lastName": "MySurname",
        "displayName": "CallMeMYF",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e5f1c4f253c820001877839",
            "name": "MyTenant",
            "code": "testtenantmh",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

创建用户

使用下表中列出的方法创建用户。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/users`	创建新用户：	无

所需请求正文属性： `username` ， `tenant_id` ， `tenancies` ， `provider`

- 请求正文示例： *

```
{
  "username": "MyUser",
  "password": "mypassword",
  "firstName": "My",
  "lastName": "User",
  "displayName": "CallMeMyUser",
  "email": "user@example.com",
  "phone": "string",
  "profileImageURL": "string",
  "tenant_id": "5e7c3af7aab46c00014ce877",
  "tenancies": [
    {
      "tenant_id": "5e7c3af7aab46c00014ce877",
      "role_name": "admin"
    }
  ],
  "provider": "local",
  "provider_data": {
    "email": "user@example.com",
    "member_of": "string"
  }
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed6f463129e5d000102f7e1",
        "username": "MyUser",
        "firstName": "My",
        "lastName": "User",
        "displayName": "CallMeMyUser",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e7c3af7aab46c00014ce877",
            "name": "MyTenant",
            "code": "mytenantcode",
            "role_name": "admin"
          }
        ]
      }
    ]
  }
}
```


按 ID 修改用户

使用下表中列出的方法按用户 ID 修改用户。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/users/{id}`	修改由用户 ID 标识的用户。您可以修改用户名，显示名称，密码，电子邮件地址，电话号码， 配置文件映像 URL 和租户详细信息。	id （string）：用户的唯一标识符。

所需的请求正文属性： none

- 请求正文示例： *

```
{
  "password": "MyNewPassword",
   "firstName": "MyFirstName",
   "lastName": "MySurname",
   "displayName": "CallMeMYF",
   "email": "user@example.com",
   "phone": "string",
  "profileImageURL": "string",
  "tenant_id": "5e5f1c4f253c820001877839",
  "tenancies": [
    {
      "tenant_id": "5e5f1c4f253c820001877839",
      "role_name": "user"
    }
  ]
}
```

- 响应正文示例：*

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61aa814559c20001df1a5f",
        "username": "MyName",
        "firstName": "MyFirstName",
        "lastName": "MySurname",
        "displayName": "CallMeMYF",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e5f1c4f253c820001877839",
            "name": "MyTenant",
            "code": "testtenantmh",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

按 ID 删除用户

使用下表中列出的方法按 ID 删除用户。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/users/ {name} `	删除此 ID 标识的用户。	`id` （ `string` ）：用户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

分区

使用下表中列出的方法创建，修改和删除分区。有关可用于检索区域的 API ，请参见（使用者）管理 API 。

HTTP 方法	路径	Description
发布	` v2.1/zones`	创建新分区。
PUT	` /v2.1/zones/ ｛ name ｝ `	修改分区的详细信息。
d删除	` /v2.1/zones/ ｛ name ｝ `	删除分区。

分区属性

下表列出了分区属性。

属性	Type	Description
id	string	分区的唯一标识符。
名称	string	分区名称。
问题描述`	string	分区的问题描述。
reGion_name	string	分区所在区域的名称。

检索所有分区

使用下表中列出的方法检索所有分区或部分分区。指定区域将仅返回属于该租户的块存储。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/zones`	检索分区。	（可选）区域名称（字符串） offset 和 limit — 请参见 "通用分页"

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 5,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 2,
    "limit": 1,
    "records": [
      {
        "id": "5ce5d919b68d3b82dc34bef2",
        "name": "au-west1-a",
        "description": "au-west1-a",
        "region": "au-west1"
      }
    ]
  }
```

按名称检索分区

使用下表中列出的方法按分区名称检索分区。

HTTP 方法	路径	Description	Parameters
获取	`/v2.1/zones/{name}`	按名称检索分区。	`Name` （ `string` ）：分区名称

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ce5d919b68d3b82dc34bef2",
        "name": "au-west1-a",
        "description": "au-west1-a",
        "region": "au-west1"
      }
    ]
  }
}
```

创建分区

使用下表中列出的方法创建分区。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/zones`	在区域中创建新分区。	无

所需的请求正文属性： `name` ，问题描述` ， `reGion_name`

- 请求正文示例： *

```
{
  "name": "MyZoneName",
  "description": "DescriptionOfMyZone",
  "region_name": "MyRegionName"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61741c9b64790001fe9663",
        "name": "MyZoneName",
        "description": "DescriptionOfMyZone",
        "region": "MyRegionName"
      }
    ]
  }
}
```

修改分区

使用下表中列出的方法修改分区。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/zones {name} `	修改以名称标识的分区。	name （ string ）：分区的名称。

所需的请求正文属性： none

- 请求正文示例： *

```
{
  "name": "MyZoneName",
  "description": "NewDescriptionOfMyZone"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61741c9b64790001fe9663",
        "name": "MyZoneName",
        "description": "NewDescriptionOfMyZone",
        "region": "MyRegionName"
      }
    ]
  }
}
```

删除分区

使用下表中列出的方法删除分区。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/zones ｛ name ｝ `	删除通过名称标识的单个分区。必须先删除区域中的所有存储资源。	name （ string ）：分区的名称。

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *

成功删除后，不会返回任何内容。

regions

使用下表中列出的方法创建，修改和删除区域。有关可用于检索区域的 API ，请参见（使用者）管理 API 。

HTTP 方法	路径	Description
获取	` v2.1/Regions`	获取区域。
获取	` /v2.1/Regies/ ｛name｝ `	按名称获取区域。
发布	` v2.1/Regions`	创建新区域。
`PUT`	` /v2.1/Regies/ ｛name｝ `	修改区域的详细信息。
`d`删除	` /v2.1/Regies/ ｛name｝ `	删除区域。

区域属性

下表列出了区域属性。

属性	Type	Description
`id`	string	区域的唯一标识符。
名称	string	区域名称。
问题描述`	string	区域的问题描述。

检索区域

使用下表中列出的方法检索所有区域或部分区域。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/Regions`	检索区域。	`offset` 和 `limit` —请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 4,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 1,
    "records": [
      {
        "id": "5e7bf44daab46c00014ce77f",
        "name": "au-east8",
        "description": "This is the new region description",
        "zones": []
      }
    ]
  }
}
```


按名称检索区域

使用下表中列出的方法按名称检索区域。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/Regies/ ｛name ｝ `	按名称检索区域。	`name` （ `string` ）：区域名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "au-east1",
        "description": "string",
        "zones": [
          "au-east1-a"
        ]
      }
    ]
  }
}
```

创建区域

使用下表中列出的方法创建区域。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/Regions`	创建新区域。	无

所需的请求正文属性： `name`

- 请求正文示例： *

```
{
  "name": "MyRegionName",
  "description": "DescriptionOfMyRegion"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e616f849b64790001fe9658",
        "name": "MyRegionName",
        "Description": "DescriptionOfMyRegion",
        "user_id": "5bbee380a2df7a04d43acaee",
        "created": "0001-01-01T00:00:00Z",
        "tags": null
      }
    ]
  }
}
```

修改区域

使用下表中列出的方法修改区域。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/Regies/ ｛name｝ `	修改以名称标识的区域。您可以更改区域的名称和问题描述。	name （ string ）：区域的名称。

所需的请求正文属性： none

- 请求正文示例： *

```
{
  "name": "MyRegionName",
  "description": "NewDescriptionOfMyRegion"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e616f849b64790001fe9658",
        "name": "MyRegionName",
        "description": "NewDescriptionOfMyRegion",
        "zones": []
      }
    ]
  }
}
```

删除区域

使用下表中列出的方法删除区域。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/Regions ｛name ｝ `	删除使用名称标识的单个区域。必须先删除某个区域中的所有分区。	`Name （ string ）：` 区域的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

ONTAP 集群

使用下表中列出的方法检索，创建，修改和删除 ONTAP 集群。

HTTP 方法	路径	Description
获取	` /v2.1/ontapclusters`	检索所有 ONTAP 集群。
获取	` /v2.1/ontapclusters/ { id } `	按 ID 检索 ONTAP 集群。
发布	` /v2.1/ontapclusters`	创建新的 ONTAP 集群。
PUT	` /v2.1/ontapclusters/ { id } `	按 ID 更新 ONTAP 集群清单。
d删除	` /v2.1/ontapclusters/ { id } `	删除 ONTAP 集群。

ONTAP 集群属性

下表列出了 ONTAP 集群的属性。

属性	Type	Description
id	string	ONTAP 集群的唯一标识符。
名称	string	ONTAP 集群名称。
uuid	string	ONTAP 集群通用唯一标识符（UUID）。
mantioning_IP	string	ONTAP 集群管理 IPv4 地址。
用户名	string	ONTAP 集群名称
密码	string	ONTAP 集群密码
provisioning_state	string	标识集群是否可用于配置操作。选项包括： • 打开 • 已关闭
data_network_ip_cidr	string	子网的 CIDR 表示法。
data_network_default_gateway	string	IPv4 地址。

属性	Type	Description

data_network_ports	–	ONTAP 集群数据网络端口列表。
		
		``
		For example:
		[
		  {
		  "node_name": "dev-ots-per01-01",
		  "port_name": "e0c-120",
		  "parent_port": "e0c"
		  }
		]
		``

intercluster_LIF	–	ONTAP 集群集群间 LIF 。
		
		``
		For example:
		[
		  {
		  "name": "peer1",
		  "node": "aff-01",
		  "port": "a0a-103",
		  "address": "10.128.113.232",
		  "netmask": "255.255.255.0"
		  },
		{
		  "name": "peer2",
		  "node": "aff-02",
		  "port": "a0a-103",
		  "address": "10.128.113.233",
		  "netmask": "255.255.255.0"
		  }
		]
		``

属性	Type	Description
sVM_root_service_level	string	ONTAP 集群 Storage Virtual Machine （ SVM ）根服务级别名称。适用值包括 " 标准 " ， " 极速 " 或 " 高级 " 。默认情况下，此服务级别会分配给在集群下创建的所有 SVM 。集群应具有所述服务级别的关联聚合。
分区	string	分区名称。
ssubscription_number	string	Zuora 订阅。
serves_available	–	列出可用服务及其状态。例如： `{ "fcp": false, "iscsi": true, "nas": true }`
data_fcp_ports	–	已启用 FCP 的 ONTAP 集群的节点和端口列表。例如： `[ { "node_name": "aff-01", "port_name": "0g" }, { "node_name": "aff-01", "port_name": "0h" ]`
is_mcc	布尔值	指示集群是否已启用 MetroCluster 。默认值为 False 。
mcc_partner_cluster	string	MetroCluster 对中当前集群的配对集群的标识符。如果为集群启用了 MetroCluster ，则为必需项。

检索所有 ONTAP 集群

使用以下列出的方法检索所有 ONTAP 集群或部分 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ontapclusters`	检索所有 ONTAP 集群。	offset 和 limit —请参见 "通用分页"

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 5,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 3,
    "limit": 2,
    "records": [
      {
        "id": "5c5bb9f16680a7002a5f7450",
        "name": "dev-ots-per01",
        "region": "au-west1",
        "zone": "au-west1-a",
        "uuid": "63053baa-ada4-11ea-b197-005056a4c0ef",
        "management_ip": "10.128.115.173",
        "username": "admin",
        "services_available": {
          "fcp": false,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "dev-ots-per01-01",
            "port_name": "e0c-120",
            "parent_port": "e0c"
          }
        ],
        "data_network_ip_cidr": "10.96.120.0/24",
        "data_network_default_gateway": "10.96.120.1",
```

```
        "svm_root_service_level": "performance",
        "intercluster_lifs": [
          {
            "name": "dev-ots-per01-01-icl01",
            "node": "dev-ots-per01-01",
            "port": "e0b",
            "address": "10.128.115.144",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00003875",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": []
      },
      {
        "id": "5eaf5249f038943eb46b6608",
        "name": "aff",
        "region": "au-east1",
        "zone": "au-east1-b",
        "uuid": "62d649d2-07a1-11e6-9549-00a0985c0dcb",
        "management_ip": "10.128.113.69",
        "username": "admin",
        "services_available": {
          "fcp": true,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "aff-01",
            "port_name": "a0a-2000",
            "parent_port": "a0a"
          },
          {
            "node_name": "aff-02",
            "port_name": "a0a-2000",
            "parent_port": "a0a"
          }
        ],
        "data_network_ip_cidr": "10.50.50.0/24",
        "data_network_default_gateway": "10.50.50.1",
        "svm_root_service_level": "premium",
        "intercluster_lifs": [
          {
            "name": "peer1",
```

```
            "node": "aff-01",
            "port": "a0a-103",
            "address": "10.128.113.232",
            "netmask": "255.255.255.0"
          },
          {
            "name": "peer2",
            "node": "aff-02",
            "port": "a0a-103",
            "address": "10.128.113.233",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00004635",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": [
          {
            "node_name": "aff-01",
            "port_name": "0g"
          },
          {
            "node_name": "aff-01",
            "port_name": "0h"
          },
          {
            "node_name": "aff-02",
            "port_name": "0g"
          },
          {
            "node_name": "aff-02",
            "port_name": "0h"
          }
        ],
        "is_mcc": false,
        "created": "1995-09-07T10:40:52Z"
      }
    ]
  }
}
```


按 **ID** 检索 **ONTAP** 集群

使用下面列出的方法按 ID 检索 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ontapclusters/ {id} `	检索 ID 标识的 ONTAP 集群。	`id` （ `string` ）：ONTAP 集群的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5c5bb9f16680a7002a5f7450",
        "name": "dev-ots-per01",
        "region": "au-west1",
        "zone": "au-west1-a",
        "uuid": "63053baa-ada4-11ea-b197-005056a4c0ef",
        "management_ip": "10.128.115.173",
        "username": "admin",
        "services_available": {
          "fcp": false,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "dev-ots-per01-01",
            "port_name": "e0c-120",
            "parent_port": "e0c"
          }
        ],
        "data_network_ip_cidr": "10.96.120.0/24",
        "data_network_default_gateway": "10.96.120.1",
```

```
        "svm_root_service_level": "performance",
        "intercluster_lifs": [
          {
            "name": "dev-ots-per01-01-icl01",
            "node": "dev-ots-per01-01",
            "port": "e0b",
            "address": "10.128.115.144",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00003875",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": [],
        "is_mcc": false,
        "created": "1995-09-07T10:40:52Z"
      }
    ]
  }
}
```


创建 **ONTAP** 集群

使用下表中的 API 创建 ONTAP 集群。

创建 ONTAP 集群时始终启用 iSCSI 服务。此外，如果基础架构支持 FCP 服务，也可以启用该服务。

HTTP 方法	路径	**Description**	**Parameters**
发布	` /v2.1/ontapclusters`	创建 ONTAP 集群。	无

所需的请求正文属性： `name` ， `uuid` ， `management_IP` ， `username` ， `password` ， `data_network_ip_cidr` ， `data_network_default_gateway` ， `intercluster_lifs` ， `zone`

如果启用了 FCP （使用 `services _available` FCP 属性），则需要 `data_fcp_ports `。

如果 `is_mcc` 为 true ，则需要 `mcc_partner_cluster` 。

- 请求正文示例： *

```
{
  "name": "clustername",
  "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
  "management_ip": "10.128.112.165",
  "username": "admin",
  "password": "ClusterPassword",
  "provisioning_state": "open",
  "data_network_ip_cidr": "10.96.112.0/24",
  "data_network_default_gateway": "10.96.112.1",
  "data_network_ports": [
    {
      "node_name": "clustername-01",
      "port_name": "e0c-112",
      "parent_port": "e0c"
    }
  ],
  "intercluster_lifs": [
    {
      "name": "clustername-01-icl01",
      "node": "clustername-01",
      "port": "e0b",
      "address": "10.128.112.222",
      "netmask": "255.255.255.0"
    }
  ],
  "svm_root_service_level": "extreme",
  "zone": "MyZone",
  "subscription_number": "",
  "services_available": {
    "fcp": false,
    "iscsi": true,
    "nas": true
  },
  "data_fcp_ports": [
  ],
  "is_mcc": false,
}
```


- 响应正文示例： *


```
{
    "status": {
        "user_message": "Okay. New resource created.",
        "verbose_message": "",
```

```
        "code": 201
    },
    "result": {
        "returned_records": 1,
        "records": [
            {
                "id": "5ef155b8f5591100010a75c5",
                "name": "clustername",
                "region": "MyRegion",
                "zone": "MyZone",
                "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
                "management_ip": "10.128.112.165",
                "username": "admin",
                "services_available": {
                    "fcp": false,
                    "iscsi": true,
                    "nas": true
                },
                "provisioning_state": "open",
                "data_network_ports": [
                    {
                        "node_name": "clustername-01",
                        "port_name": "e0c-112",
                        "parent_port": "e0c"
                    }
                ],
                "data_network_ip_cidr": "10.96.112.0/24",
                "data_network_default_gateway": "10.96.112.1",
                "svm_root_service_level": "extreme",
                "intercluster_lifs": [
                    {
                        "name": "clustername-01-icl01",
                        "node": "clustername-01",
                        "port": "e0b",
                        "address": "10.128.112.222",
                        "netmask": "255.255.255.0"
                    }
                ],
                "subscription_number": "",
                "created": "2020-06-23T01:07:04.563Z",
                "data_fcp_ports": [],
                "is_mcc": false,
                "mcc_partner_cluster": "5d2fb0fb4f47df00015274e3",
                "created": "1995-09-07T10:40:52Z"
            }
        ]
```

```
      }
}
```

修改 ONTAP 集群

使用以下列出的方法修改 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
`PUT`	`/v2.1/ontapclusters/{id}`	修改 ID 标识的 ONTAP 集群的详细信息。	`id` （ `string` ）：ONTAP 集群的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "clustername",
  "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
  "management_ip": "10.128.112.165",
  "username": "admin",
  "password": "ClusterPassword",
  "provisioning_state": "open",
  "data_network_ip_cidr": "10.96.112.0/24",
  "data_network_default_gateway": "10.96.112.1",
  "data_network_ports": [
    {
      "node_name": "dev-ots-syd01-01",
      "port_name": "e0c-112",
      "parent_port": "e0c"
    }
  ],
  "intercluster_lifs": [
    {
      "name": "dev-ots-syd01-01-icl01",
      "node": "dev-ots-syd01-01",
      "port": "e0b",
      "address": "10.128.112.222",
      "netmask": "255.255.255.0"
    }
  ],
  "svm_root_service_level": "standard",
  "zone": "MyZone",
  "subscription_number": "",
  "services_available": {
    "fcp": false,
    "iscsi": true,
    "nas": false
  },
  "data_fcp_ports": [
  ]
}
```


- 响应正文示例： *


```
{
    "status": {
        "user_message": "Okay. Accepted for processing.",
        "verbose_message": "",
        "code": 202
```

```
    },
    "result": {
        "returned_records": 1,
        "records": [
            {
                "id": "5ef155b8f5591100010a75c5",
                "name": "clustername",
                "region": "MyRegion",
                "zone": "MyZone",
                "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
                "management_ip": "10.128.112.165",
                "username": "admin",
                "services_available": {
                    "fcp": false,
                    "iscsi": true,
                    "nas": true
                },
                "provisioning_state": "open",
                "data_network_ports": [
                    {
                        "node_name": "dev-ots-syd01-01",
                        "port_name": "e0c-112",
                        "parent_port": "e0c"
                    }
                ],
                "data_network_ip_cidr": "10.96.112.0/24",
                "data_network_default_gateway": "10.96.112.1",
                "svm_root_service_level": "standard",
                "intercluster_lifs": [
                    {
                        "name": "dev-ots-syd01-01-icl01",
                        "node": "dev-ots-syd01-01",
                        "port": "e0b",
                        "address": "10.128.112.222",
                        "netmask": "255.255.255.0"
                    }
                ],
                "subscription_number": "",
                "created": "2020-06-23T01:07:04.563Z",
                "data_fcp_ports": [],
                "is_mcc": false,
                "mcc_partner_cluster": "5d2fb0fb4f47df00015274e3",
                "created": "1995-09-07T10:40:52Z"
            }
        ]
    }
```

```
}
```

删除 ONTAP 集群

使用下表中列出的方法删除 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/ontapclusters/ ｛id｝ `	删除 ID 标识的 ONTAP 集群。	id （ string ）：ONTAP 集群的唯一标识符。

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

StorageGRID 实例

使用下表中列出的方法设置和管理 StorageGRID 实例以支持对象存储。

HTTP 方法	路径	Description
获取	` /v2.1/storageggridinstances`	检索 StorageGRID 实例。
发布	` /v2.1/storageggridinstances`	创建新的 StorageGRID 实例以托管对象存储。
获取	` /v2.1/storageggridinstances/ ｛id｝ `	按 ID 检索 StorageGRID 实例。
PUT	` /v2.1/storageggridinstances/ ｛id｝ `	更新 StorageGRID 实例。

StorageGRID 实例属性

下表列出了 StorageGRID 实例属性。

属性	Type	Description
id	string	StorageGRID 实例的唯一标识符。
名称	string	StorageGRID 实例名称。
admin_rest_uri	字符串（ $uri ）	StorageGRID 管理节点端点。

属性	Type	Description
`s 3_Endpoints`	字符串（$uri）	StorageGRID 端点。例如： [<code>"<a href="https://s3.examplegrid.com"" class="bare">https://s3.examplegrid.com"</a>， "https://s3.location.company.com"</code>[]]
`grid_username`	string	StorageGRID 用户名。
`grid_password`	string	StorageGRID 密码。
`tenant_username`	string	StorageGRID 租户用户名。
`tenant_password`	string	StorageGRID 租户密码。
`ssubscription_number`	string	Zuora 订阅编号。

检索 **StorageGRID** 实例

使用下表中列出的方法检索 StorageGRID 实例。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/storageggridinstances`	检索 StorageGRID 实例。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e3ba2840271823644cb8ab6",
        "name": "NSE StorageGRID Dev1",
        "user_id": "5bbee380a2df7a04d43acaee",
        "admin_rest_uri": "https://sggmi-dev.dev.ausngs.netapp.au",
        "s3_endpoints": [
          "https://sgs3.dev.ausngs.netapp.au"
        ],
        "subscription_number": "A-S00004566"
      }
    ]
  }
}
```


按 ID 检索 StorageGRID 实例

使用下表中列出的方法按 ID 检索 StorageGRID 实例。

HTTP 方法	路径	Description	Parameters
获取	`/v2.1/storageggridinstances/｛id｝`	按 ID 检索 StorageGRID 实例。	`id （ string ）：` StorageGRID 实例的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e3ba2840271823644cb8ab6",
        "name": "NSE StorageGRID Dev1",
        "user_id": "5bbee380a2df7a04d43acaee",
        "admin_rest_uri": "https://sggmi-dev.dev.ausngs.netapp.au",
        "s3_endpoints": [
          "https://sgs3.dev.ausngs.netapp.au"
        ],
        "subscription_number": "A-S00004566"
      }
    ]
  }
}
```


按 ID 创建 StorageGRID 实例

使用下表中列出的方法按 ID 创建 StorageGRID 实例。

HTTP 方法	路径	Description	Parameters
发布`	` /v2.1/storageggridinstances/｛id｝`	按 ID 检索 StorageGRID 实例。	`id` （字符串）：StorageGRID 实例的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "Grid1",
  "admin_rest_uri": "https://examplegrid.com",
  "s3_endpoints": [
    "https://s3.examplegrid.com",
    "https://s3.location.company.com"
  ],
  "grid_username": "root",
  "grid_password": "string",
  "tenant_username": "root",
  "tenant_password": "string",
  "subscription_number": "A-S00003969"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "Grid1",
        "admin_rest_uri": "https://examplegrid.com",
        "user_id": "5d2fb0fb4f47df00015274e3",
        "s3_endpoints": [
          "https://s3.examplegrid.com",
          "https://s3.location.company.com"
        ],
        "subscription_number": "A-S00003969"
      }
    ]
  }
}
```

按 ID 修改 StorageGRID 实例

使用下表中列出的方法按 ID 修改 StorageGRID 实例。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/storageggridinstances/｛id｝`	按 ID 修改 StorageGRID 实例。	id （ string ）：StorageGRID 实例的唯一标识符。

所需请求正文属性： none

- 请求正文示例： *

```
{
  "name": "Grid1",
  "admin_rest_uri": "https://examplegrid.com",
  "s3_endpoints": [
    "https://s3.examplegrid.com",
    "https://s3.location.company.com"
  ],
  "grid_username": "root",
  "grid_password": "string",
  "tenant_username": "root",
  "tenant_password": "string",
  "subscription_number": "A-S00003969"
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "Grid1",
        "admin_rest_uri": "https://examplegrid.com",
        "user_id": "5d2fb0fb4f47df00015274e3",
        "s3_endpoints": [
          "https://s3.examplegrid.com",
          "https://s3.location.company.com"
        ],
        "subscription_number": "A-S00003969"
      }
    ]
  }
}
```


服务级别

使用下表中列出的方法检索，创建，修改和删除服务级别。

HTTP 方法	路径	Description
获取	` /v2.1/ServiceLevels`	检索所有服务级别。
获取	` /v2.1/ServiceLevers/ { id } `	按 ID 检索服务级别。
发布	` /v2.1/ServiceLevels`	创建新的服务级别。
PUT	` /v2.1/ServiceLevers/ { id } `	按 ID 修改服务级别详细信息。
d删除	` /v2.1/ServiceLevers/ { id } `	删除服务级别。

服务级别属性

下表列出了服务级别属性。

属性	Type	Description
id	string	服务级别的唯一标识符。
名称	string	服务级别名称。

属性	Type	Description
问题描述`	string	服务级别的问题描述。
policy_name	string	服务级别服务质量（ QoS ）名称。允许的值： NSE_value ， nse_standard ， nse_performance 和 nse_Extreme 。
可用	布尔值	指示服务级别是否可供使用。
祖父系	布尔值	指示服务级别是否已停用。
pease_IOPS_per_TB	整型	每 TiB 的最大可能 IOPS 。
expected_IOPS_per_TB	整型	每 TiB 的最小预期 IOPS 。
abory_min_IOPS	整型	当预期 IOPS 小于此值时用作覆盖的绝对最小 IOPS 。
pease_IOPS_allocation	string	峰值 IOPS 分配。允许值：allocated_space 和 used_space 。
io_block_size_KB	整型	I/O 块大小（ KiB ）。
min_size_GB	整型	最小大小（ GiB ）。
max_size_GB	整型	最大大小（ GiB ）。
max_pease_IOPS	整型	服务级别的最大峰值 IOPS 。
max_expected_IOPS	整型	最大预期 IOPS 。
autogrow_max_percent	整型	自动增长最大百分比。

属性	Type	Description
ontap_aggregates	–	ONTAP 聚合的列表。ONTAP 聚合包括： • cluster_UIID ： ONTAP 集群 UUID • aggr_name ： ONTAP 聚合名称 • aggr_uuid ： ONTAP 聚合 UUID • node_name ： ONTAP 集群节点名称 例如： "ontap_aggregates": [{ "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "aggr_name": "string", "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "node_name": "node01" }]

属性	Type	Description
primary_volume_defaults	–	• snapshot_auto_delete_target_free_space （整型）：此选项指定必须停止自动删除 Snapshot 副本的可用空间百分比。 • auto_size_mode （字符串）：卷的自动调整大小模式。允许的值： off ， grow ， grow_shrink ，例如： `"primary_volume_defaults": { "snapshot_auto_delete_target_free_space": 3, "auto_size_mode": "grow_shrink"`

检索服务级别

使用下表中列出的方法检索所有服务级别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ServiceLevels`	检索所有服务级别。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 3 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 3,
    "records": [
```

```
      {
        "name": "standard",
        "description": "Best suited for general purpose workloads",
        "slo": "1000IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 40960,
        "min_iops": 100,
        "peak_iops_per_tb": 1000,
        "expected_iops_per_tb": 700,
        "max_peak_iops": 40000,
        "max_expected_iops": 28000,
        "max_peak_throughput": 1250,
        "max_expected_throughput": 875
      },
      {
        "name": "extreme",
        "description": "Best suited for performance-critical workloads",
        "slo": "12000IOPS/TB",
        "min_size": 91625968981,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 500,
        "peak_iops_per_tb": 12000,
        "expected_iops_per_tb": 8000,
        "max_peak_iops": 120000,
        "max_expected_iops": 60000,
        "max_peak_throughput": 3750,
        "max_expected_throughput": 1875
      },
      {
        "name": "premium",
        "description": "Best suited for databases and high performance
workloads",
        "slo": "4000IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 300,
        "peak_iops_per_tb": 4000,
        "expected_iops_per_tb": 3000,
        "max_peak_iops": 40000,
        "max_expected_iops": 30000,
```

```
        "max_peak_throughput": 1250,
        "max_expected_throughput": 937
      }
    ]
  }
}
```

按名称检索服务级别

使用下表中列出的方法按名称检索服务级别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ServiceLevers/ { name } `	按名称检索服务级别。	`name` （ `string` ）：服务级别的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "name": "premium",
        "description": "Best suited for databases and high performance
workloads",
        "slo": "4096IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 300,
        "peak_iops_per_tb": 4096,
        "expected_iops_per_tb": 3000,
        "max_peak_iops": 40000,
        "max_expected_iops": 30000,
        "max_peak_throughput": 1250,
        "max_expected_throughput": 937
      }
    ]
  }
}
```


创建服务级别

使用下表中列出的方法创建服务级别。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/ServiceLevels`	创建服务级别。	无

所需的请求正文属性： `name` ， `policy_name`

- 请求正文示例： *

```
{
  "name": "MyServiceLevelName",
  "description": "My new service level description",
  "policy_name": "nse_value",
  "available": true,
  "grandfathered": false,
  "peak_iops_per_tb": 1000,
  "expected_iops_per_tb": 700,
  "absolute_min_iops": 100,
  "peak_iops_allocation": "allocated_space",
  "io_block_size_kb": 32,
  "min_size_gb": 10,
  "max_size_gb": 40960,
  "max_peak_iops": 20000,
  "max_expected_iops": 5000,
  "autogrow_max_percent": 3,
  "ontap_aggregates": [
    {
      "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "aggr_name": "string",
      "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "node_name": "node01"
    }
  ],
  "primary_volume_defaults": {
    "snapshot_auto_delete_target_free_space": 3,
    "auto_size_mode": "grow_shrink"
  }
}
```


- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "name": "MyServiceLevelName",
        "description": "My new service level description",
        "slo": "1000IOPS/TB",
        "min_size": 0,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 40960,
        "min_iops": 100,
        "peak_iops_per_tb": 1000,
        "expected_iops_per_tb": 700,
        "max_peak_iops": 20000,
        "max_expected_iops": 5000,
        "max_peak_throughput": 625,
        "max_expected_throughput": 156
      }
    ]
  }
}
```


修改服务级别

使用下表中列出的方法修改服务级别。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/ServiceLevers/ { name } `	修改服务级别的详细信息。	name （ string ）：服务级别的名称。

所需的请求正文属性： none

- 请求正文示例： *

```
{
  "name": "MyNewServiceLevelName",
  "description": "Service level description",
  "policy_name": "nse_value",
  "available": false,
  "grandfathered": false,
  "peak_iops_per_tb": 1000,
  "expected_iops_per_tb": 700,
  "absolute_min_iops": 100,
  "peak_iops_allocation": "allocated_space",
  "io_block_size_kb": 32,
  "min_size_gb": 10,
  "max_size_gb": 40960,
  "max_peak_iops": 20000,
  "max_expected_iops": 5000,
  "autogrow_max_percent": 3,
  "ontap_aggregates": [
    {
      "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "aggr_name": "string",
      "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "node_name": "node01"
    }
  ],
  "primary_volume_defaults": {
    "snapshot_auto_delete_target_free_space": 3,
    "auto_size_mode": "grow_shrink"
  }
}
```

- 响应正文示例： *

```
TBA
```

按 ID 删除服务级别

使用下表中列出的方法按 ID 删除服务级别。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/ServiceLevers/ { name } `	删除 ID 标识的服务级别。	`name` （ `string` ）：服务级别的名称。

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

服务请求

使用下表中列出的方法创建和检索服务请求。

HTTP 方法	路径	Description
获取	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequests`	检索服务请求。
获取	` /v2.1/tenant/ ｛ tenant_id ｝ / 服务要求 / ｛ id ｝ `	按 ID 检索服务请求。
发布	` /v2.1/tenant/ ｛ tenant_id ｝ /servicereques/`	创建服务请求。
获取	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequies/categories`	检索服务请求类别。

服务请求属性

下表列出了服务请求属性。

属性	Type	Description
`ID`	string	服务请求的标识符。例如： `sQ0035952014` 。
`sSubject`	string	服务请求的主题。
注释	string	对服务请求的注释
类别	string	请求的类别：备份，灾难恢复，技术，其他或订阅。
优先级	string	服务请求的优先级：极低，低，正常，高或紧急。
`ssubscription`	string	Zuora 订阅编号。
承诺	–	订阅承诺详细信息，包括： • 服务级别（字符串）：服务级别：标准，高级或至尊。 • new_commitment （整型）：需要新的总容量。

属性	Type	Description
分区	string	分区名称。
`ssubtenant_id`	string	子租户标识符。
`storage_object_type`	string	存储对象类型：文件服务器或块存储。
`storage_object_id`	string	存储对象的标识符。例如：`5d2fb0fb4f47df00015274e3`
`sservice_type`	string	服务类型：文件服务，块存储或对象存储。

检索服务请求

使用下表中列出的方法检索指定租户的服务请求类别。

HTTP 方法	路径	Description	Parameters
获取`	` /v2.1/tenant/ { tenant_id } /servicerequests`	检索服务请求。	`tenant_id`：（可选）返回指定租户的服务请求。`offset` 和 `limit` — 请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 34,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 6,
    "limit": 1,
    "records": [
      {
        "id": "SRQ0035952014",
        "subject": "DR Failover - fileserver",
        "description": "catgory:Disaster Recovery Failover \n   subtenant:
DefaultSubtenants2 \n  region: au-east2 \n zone: au-east2-a \n
fileserver: Demotsysserv1 \n tenant:MyOrg \n comments:comments",
        "priority": "Urgent",
        "status": "New",
        "createdDate": "2020-05-22T04:23:12+0000",
        "updatedDate": "2020-05-22T04:23:12+0000"
      }
    ]
  }
}
```


按 ID 检索服务请求

使用下表中列出的方法按服务请求 ID 检索服务请求。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/tenant/ ｛tenant_id｝ / 服务要求 / ｛id｝ `	按 ID 检索服务请求。	• `tenant_id`： 租户 ID • `id`： 服务请求 ID 例如：SRQ0035952014

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "SRQ0035952014",
        "subject": "DR Failover - fileserver",
        "description": "catgory:Disaster Recovery Failover \n   subtenant:
DefaultSubtenants2 \n  region: au-east2 \n zone: au-east2-a \n
fileserver: Demotsysserv1 \n tenant:MyOrg \n comments:comments",
        "priority": "Urgent",
        "status": "New",
        "createdDate": "2020-05-22T04:23:12+0000",
        "updatedDate": "2020-05-22T04:23:12+0000"
      }
    ]
  }
}
```

创建服务请求

使用下表中列出的方法创建服务请求。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequies/categories `	创建服务请求。	`tenant_id`：租户标识符。

所需请求正文属性：所需属性取决于服务请求的类别。下表列出了请求正文属性。

类别	Required
订阅。	`ssubscription` and `commitment`

类别	Required
灾难恢复	storage_object_type ， ssubtenant_id 和 storage_object_id
技术	ssubtenant_id 和 sservice_type 如果 sservice_type 是文件服务或块存储，则需要区域。
其他	分区

- 请求正文示例： *

```
{
  "subject": "string",
  "comment": "string",
  "category": "subscription",
  "priority": "Normal",
  "subscription": "A-S00003969",
  "commitment": {
    "service_level": "standard",
    "new_commitment": 10
  },
  "zone": "au-east1-a",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "storage_object_type": "fileserver",
  "storage_object_id": "5d2fb0fb4f47df00015274e3",
  "service_type": "File Services"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "string",
        "subject": "string",
        "description": "string",
        "status": "New",
        "priority": "Normal",
        "createdDate": "2020-05-12T03:18:25+0000",
        "UpdatedDate": "2020-05-12T03:18:25+0000"
      }
    ]
  }
```

检索服务请求类别

下表列出了指定租户的检索服务请求类别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/tenant/ ｛ tenant_id } /servicerequies/categories `	检索服务请求类别。	`tenant_id` ： （可选）返回指定租户的服务请求。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 5 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 5,
    "records": [
      {
        "key": "dr",
        "value": "Disaster Recovery Failover"
      },
      {
        "key": "technical",
        "value": "Technical Issue"
      },
      {
        "key": "other",
        "value": "Other"
      },
      {
        "key": "subscription",
        "value": "Subscription Management"
      },
      {
        "key": "backup",
        "value": "Backup Restore"
      }
    ]
  }
}
```


作业

使用作业 API 查看作业详细信息。有关使用作业的详细信息，请参见作业对象 API 和异步进程。

检索作业

使用下表中列出的方法检索所有作业或部分作业。指定 `tenant_id` 将仅返回属于该租户的作业。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/jobs`	检索作业。	`tenant_id`：（可选）返回属于指定租户的块存储。`offset` 和 `limit` — 请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 2625,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 1,
    "records": [
      {
        "id": "5ed72c8c6342e90001439d54",
        "action": "create",
        "job_summary": "Create request is successfully submitted",
        "created": "2020-06-03T04:52:28.478Z",
        "updated": "2020-06-03T04:52:32.636Z",
        "object_id": "5ed72c8c6342e90001439d55",
        "object_type": "sg_buckets",
        "object_name": "test1234",
        "status": "successful",
        "status_detail": "Creation of s3 bucket 'test1234' completed
successfully.",
        "last_error": "",
        "user_id": "5e85225af038943eb4b74684",
        "job_tasks": [
          {
            "id": "5ed72c8c6342e90001439d57",
```

```
            "job_id": "5ed72c8c6342e90001439d54",
            "action": "create",
            "driver": "storagegrid_ansible",
            "object_id": "5ed72c8c6342e90001439d55",
            "object_type": "sg_buckets",
            "resource_type": "sg_bucket",
            "status": "successful",
            "status_detail": "Worker completed task successfully.",
            "last_error": "",
            "user_id": "5e85225af038943eb4b74684",
            "request_payload": {
              "grid_account_id": "05336917559886003354",
              "grid_admin_base_url": "https://sggmi-
dev.dev.ausngs.netapp.au/api/v3",
              "org_password": "netapp01",
              "org_username": "root",
              "s3_bucket_name": "test1234"
            }
          }
        ]
      }
    ]
  }
}
```


按作业 **ID** 检索作业

使用下表中列出的方法按 ID 检索作业。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/jobs/｛id｝`	按 ID 检索作业。	`id` （ `string` ）：作业的唯一标识符。

所需的请求正文属性： 作业标识符

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
```

```
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e66f18e09a74c0001b89640",
        "action": "create",
        "job_summary": "Create S3 bucket for Sub Tenant",
        "created": "2020-03-10T01:46:54.097Z",
        "updated": "2020-03-10T01:46:57.664Z",
        "object_id": "5e66f18e09a74c0001b89641",
        "object_type": "sg_buckets",
        "status": "successful",
        "status_detail": "Creation of s3 bucket 'mys3bucket' completed
successfully.",
        "last_error": "",
        "user_id": "5bbee380a2df7a04d43acaee",
        "job_tasks": [
          {
            "id": "5e66f18e09a74c0001b89642",
            "job_id": "5e66f18e09a74c0001b89640",
            "action": "create",
            "driver": "storagegrid_ansible",
            "object_id": "5e66f18e09a74c0001b89641",
            "object_type": "sg_buckets",
            "resource_type": "sg_bucket",
            "status": "successful",
            "status_detail": "Worker completed task successfully.",
            "last_error": "",
            "user_id": "5bbee380a2df7a04d43acaee",
            "request_payload": {
              "grid_account_id": "47490102387197219062",
              "grid_admin_base_url": "https://sggmi-
dev.dev.ausngs.netapp.au/api/v3",
              "org_password": "netapp01",
              "org_username": "root",
              "s3_bucket_name": "mys3bucket"
            }
          }
        ]
      }
    ]
  }
}
```

某些 API 调用（尤其是用于添加或修改资源的 API 调用）的完成时间可能比其他调用更长。NetApp 服务引擎会异步处理这些长时间运行的请求。

发出异步运行的 API 调用后， HTTP 响应代码 202 表示此请求已成功验证并被接受，但尚未完成。此请求将作为后台任务进行处理，在对客户端进行初始 HTTP 响应后，此任务将继续运行。响应包括作业对象锁定请求，包括其唯一标识符。

正在查询与 **API** 请求关联的作业对象

HTTP 响应中返回的作业对象包含多个属性。您可以查询 state 属性以确定请求是否成功完成。作业对象可以处于以下状态之一：

- 正常
- 警告
- Partial_failures
- error

在轮询作业对象以检测任务的终端状态时，可以使用两种方法：成功或失败：

- 标准轮询请求：立即返回当前作业状态。
- 长轮询请求：当作业状态变为 normal ， error 或 partial_failures 时。

异步请求的步骤

您可以使用以下高级操作步骤完成异步 API 调用。

1. 问题描述异步 API 调用。
2. 接收表示已成功接受请求的 HTTP 响应 202 。
3. 从响应正文中提取作业对象的标识符。
4. 在环路中，等待作业对象达到终端状态 normal ， error 或 partial_failures 。
5. 验证作业的终端状态并检索作业结果。

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.